

Drone Coverage Path Planning

Lavinia Amorosi^a, Paolo Dell’Olmo^a, Justo Puerto^b, Carlos Valverde^{b,*}

^a*Department of Statistical Sciences, Sapienza University of Rome, Italy*

^b*Department of Statistical Sciences and Operational Research, University of Seville, Spain*

Abstract

We present a mixed-integer quadratic programming (MIQP) formulation for a drone coverage path planning problem. A single drone, with limited energy, must depart from a depot, cover a set of disjoint polygonal regions in the plane, and return to the depot to swap the battery and start a new tour, until all the regions are covered. Each region can be inspected at a discrete set of altitudes, each associated with a polygonal chain that defines the coverage pattern (altitude selection). A finite set of entry/exit points is associated with each region, allowing the drone to interrupt the coverage of a region and resume it later from a different point. The formulation embeds altitude selection, the geometry of the entry/exit points along the chain, and operation splitting under the endurance budget, using second-order-cone constraints for the Euclidean distances and a big-M linearization of the bilinear products between routing and distance/energy variables. We also propose an equivalent Edges-Approach reformulation that replaces vertex-visitation variables by edge-traversal variables and uses DFJ lazy subtour elimination, and a multi-phase constructive heuristic that produces feasible solutions in well under a second on all tested instances and is used to warm-start the exact solver. Two extensions are studied: splitting each coverage ring into $K \geq 1$ continuous arcs, so that entry and exit points need no longer coincide (with $K = 1$ recovering the original model), and a wind-aware energy model in real joules with precomputable per-heading airspeed costs. On a testbed of 1080 artificial instances the Vertex-Approach formulation dominates without warm-start (94% solved vs. 58%), the two formulations agree within 1% once warm-started, and the heuristic stays within 8% of the exact mean objective at every scale; one split point per ring ($K = 2$) already improves solutions by up to 25%, with no further gains beyond $K = 3$ on the tested instances.

Keywords: Drone coverage path planning, Mixed-Integer Quadratic Programming, UAV, Coverage patterns

1. Introduction

The optimization of unmanned aerial vehicle (UAV) operations is inherently multifaceted: it involves path planning, resource allocation, energy management, and collision avoidance in increasingly crowded airspaces. Determining the optimal route for a drone to cover a designated area while minimizing energy consumption and avoiding obstacles is a complex combinatorial and continuous optimization problem. When multiple drones are deployed

*Corresponding author

simultaneously, coordination and scheduling add a further layer of difficulty, demanding algorithms that can handle dynamic environmental conditions and regulatory constraints.

Drones have been successfully adopted in many contexts, including disaster management [1, 2], telecommunications for providing 5G coverage of rural zones [3], inspection of planar graphs [4, 5, 6], 3D mapping and video surveillance [7], and crop monitoring in agriculture [8]. A broader survey of civil UAV applications is given in [9].

In this paper we focus on coverage of polygonal areas in the plane by a single drone. We propose a mixed-integer quadratic programming (MIQP) formulation that jointly optimizes the altitude, the entry and exit points, and the sequence of operations under a limited energy budget. Unlike existing two-stage approaches that first plan intra-region paths and then solve an inter-region routing problem, our formulation allows the drone to interrupt the coverage of a region and resume it later, potentially reducing total energy consumption. To tackle large instances, we also develop a multi-phase constructive heuristic that provides high-quality feasible solutions in well under a second and can be used to warm-start the exact solver. The main contributions of this paper are:

- A MIQP formulation for the drone coverage path planning problem that integrates altitude selection, variable entry/exit points, and operation splitting under endurance constraints.
- An equivalent Edges-Approach reformulation that replaces vertex-visitation variables with edge-coverage variables and uses DFJ lazy subtour elimination, offering a different trade-off between constraint structure and relaxation strength.
- A multi-phase heuristic combining chain selection, region-level tour construction, greedy endurance-aware tour splitting, coordinate-descent entry-point optimization, and intra/inter-operation local search.
- A generalization to $K \geq 1$ split points per coverage ring, allowing entry and exit points to differ, with nested models ($K = 1$ is the original formulation) and a knee in solution quality at $K = 2$ –3.
- A wind-aware energy extension in real joules with precomputable per-heading costs that preserves the MIQP structure and validates that optimal tours are robust to strong wind on the tested instances.
- A computational study on a testbed of 1080 artificial instances (540 per formulation) with varying numbers of regions, altitude levels, and endurance values, comparing the exact solver with and without warm-start against the standalone heuristic and the two equivalent MIQP formulations, plus split-point sweeps and wind-validation experiments.

2. State of the art

The coverage path planning problem (CPP) has been studied widely in the literature. A comprehensive survey that includes a taxonomy of CPP algorithms and drone-specific characteristics is presented in [10]. Recent work can be grouped into three broad methodological streams.

Back-and-forth coverage patterns.. Several authors adopt variants of the back-and-forth (also known as lawnmower or boustrophedon) strategy for intra-region coverage. [11] propose an improved back-and-forth algorithm with a straight-turn mode to cover rectangular regions containing forbidden areas. [12] develop a mathematical model and heuristic for energy-efficient multi-UAV multi-region CPP, using back-and-forth patterns with Bezier-curve smoothing to reduce energy consumption from turns; their optimization proceeds in two decoupled phases (intra-region path generation then inter-region routing). Similarly, [13] generate waypoints for polygonal areas and apply smoothing to reduce flight time and memory footprint.

Two-stage and sequential approaches.. Many methods first plan coverage paths within each region and then solve a separate routing problem. [14] present a CPP method for spraying drones that disinfect areas, tested on realistic scenarios. [15] propose a two-stage method for mixed regions with limited energy: they shrink perimeters to generate optimal paths, then use dynamic programming to segment the overall path among multiple drones to satisfy energy limits. [16] develop a trajectory and velocity control approach for energy-efficient visual coverage of multiple terrestrial regions, solving the problem in four sequential stages.

Energy-aware formulations with altitude and speed control.. The works most closely related to ours jointly consider energy, altitude, and speed decisions. [16] allow the UAV to change both velocity and altitude during its tour, optimizing visual coverage quality subject to energy constraints. Our approach differs from all of the above in that we allow the drone to interrupt coverage of a region and move to another one before returning, rather than fully covering each region in a single pass. This flexibility, enabled by the edge set $\mathcal{E}^{ext}$ in our model, can lead to shorter total tours compared to the two-stage paradigm, as illustrated in Section 3.

3. Problem Description

In this paper we study a drone coverage path planning problem. Given a set of non-overlapping polygonal areas to be visited, a single drone with limited endurance, and a depot, the problem consists of finding the set of tours (operations) starting and ending at the depot that the drone must perform in order to cover all areas. We assume that every region can be fully covered in a single operation, i.e. the drone has enough endurance to depart from the depot, cover a region, and return to the depot in one go.

Let $\mathcal{R} = \{1, \dots, R\}$ be the set of regions to be covered, with $R = |\mathcal{R}|$. For each region $r \in \mathcal{R}$, the user specifies a set of polygonal chains $\mathcal{T}_r$, each at a different altitude and each defining a candidate coverage pattern. The visit of a region can be performed with interruptions: the drone may leave a region before completing its coverage and resume it from another point if doing so reduces the total energy consumption. The polygonal chain of a region can be traversed in both directions. For tractability, we assume that the number of interruptions per region is at most $R - 1 = |\mathcal{R}| - 1$.

For each region $r \in \mathcal{R}$, we index the set of $2R$ candidate points along its selected chain by $\mathcal{I}_r = \{1, \dots, 2R\}$, partitioned into four subsets:

- $\mathcal{I}_S^r = \{1, 2\}$: the two endpoints at the start of the chain (each can act as a launching or retrieving point);

- $\mathcal{I}_L^r = \{3, 5, \dots, 2R - 1\}$: the $R - 1$ launching points;
- $\mathcal{I}_R^r = \{4, 6, \dots, 2R\}$: the $R - 1$ retrieving points;
- $\mathcal{I}_T^r = \{2R + 1, 2R + 2\}$: the two endpoints at the end of the chain (each can act as a launching or retrieving point).

The launch and retrieve points come in pairs: a launch point $2k + 1$ is paired with the retrieve point $2k + 2$ for $k = 1, \dots, R - 1$.

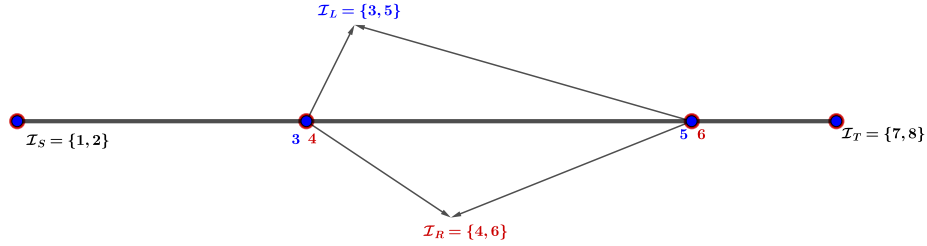


Figure 1: Indexing of the candidate points along the chain of a single region (R regions in total).

We define the graph $\mathcal{G} = (\mathcal{V} = \{v_0\} \cup \mathcal{V}', \mathcal{E} = \mathcal{E}^{int} \cup \mathcal{E}^{ext})$, where:

- $v_0 = (0, 0, 0)$ denotes the depot.
- $\mathcal{V}' = \bigcup_{r \in \mathcal{R}} \{r\} \times \mathcal{I}_r$ is the set of all candidate points across regions. It is partitioned as $\mathcal{V}' = \mathcal{V}_S \cup \mathcal{V}_L \cup \mathcal{V}_R \cup \mathcal{V}_T$, where $\mathcal{V}_S = \bigcup_{r \in \mathcal{R}} \{r\} \times \mathcal{I}_S^r$, and similarly for $\mathcal{V}_L$, $\mathcal{V}_R$, $\mathcal{V}_T$.
- $\mathcal{E}^{int} = \{(v, v') : v = (r, i), v' = (r, i + 1), \forall r \in \mathcal{R}, \forall i \in \mathcal{I}_r \setminus \{2R\}\}$ is the set of edges joining consecutive points on the same chain. It is partitioned into $\mathcal{E}_{LR}^{int}$ (forward edges) and $\mathcal{E}_{RL}^{int}$ (backward edges).
- $\mathcal{E}^{ext} = \{(v, v') : v = (r, i) \in \mathcal{V}' \setminus \mathcal{V}_R, v' = (r', i') \in \mathcal{V}' \setminus \mathcal{V}_L, r \neq r'\}$ is the set of arcs that join a non-retrieving point in one region to a non-launching point in another region.
- $\mathcal{O}$ is the set of operations, each a drone tour starting and ending at the depot v_0 .

For each region $r \in \mathcal{R}$, the set $\mathcal{T}_r$ of polygonal chains contains the candidate coverage patterns for that region. Each chain $t \in \mathcal{T}_r$ is associated with an altitude h_t and a sequence of segments $\{[Q^s, Q^{s+1}] : s \in \mathcal{S}_t\}$, where Q^s, Q^{s+1} are the endpoints of segment s and $\mathcal{S}_t = \{1, \dots, S_t\}$ is the set of segments of the chain. The selection of one chain per region is a decision variable of the model.

We assume that air density, denoted by ρ , depends on height as given by the following function (derived from the ideal gas law and the International Standard Atmosphere model):

$$\rho_h = \frac{p_0 M}{C_R T_0} \left(1 - \frac{Lh}{T_0}\right)^{\frac{gM}{C_R L} - 1},$$

where the constants are:

- $p_0 = 101325$ Pa: sea level standard atmospheric pressure;
- $T_0 = 288.15$ K: sea level standard temperature;
- $g = 9.80665$ m/s²: earth-surface gravitational acceleration;
- $L = 0.0065$ K/m: temperature lapse rate;
- $C_R = 8.31446$ J/(mol·K): universal gas constant;
- $M = 0.0289652$ kg/mol: molar mass of dry air.

We also assume that the wind has a constant direction represented by the unit vector $\vec{u}^w$. The magnitude of the wind speed depends on height following the power law of [17]:

$$\nu_h^w = \nu_{10}^w \left(\frac{h}{10}\right)^a,$$

where ν_{10}^w is the wind speed at 10 m above ground and a is the Hellmann exponent (which depends on the coastal location, terrain roughness, and air stability). The wind velocity vector at height h is then $\vec{\nu}_h^w = \nu_h^w \vec{u}^w$.

Given the wind vector at altitude h , the drone airspeed required to maintain a ground velocity $\vec{\nu}$ is

$$\vec{\nu}_h^d = \vec{\nu} - \vec{\nu}_h^w.$$

Because the drone's position and the chosen chain are decision variables, the actual flight speed along an inter-region leg is unknown before the model is solved. To avoid bilinear terms, we approximate the direction of an inter-region leg (r, r') by the unit vector from the centroid of r to the centroid of r' , giving a pre-computable ground speed $\vec{\nu}_{rr'h}^d$ (and hence energy) per region pair. The exact leg directions are recovered through the continuous position variables in the formulation.

One key feature of our problem setting is that the drone is not forced to fully explore one region before moving to another one. The edge set $\mathcal{E}^{ext}$ allows the drone to leave a region before completing its coverage and explore a different one. Other CPP contributions propose a two-stage approach, in which each region is fully covered first and then an inter-region routing problem is solved; this restriction can be suboptimal, as illustrated by the following example.

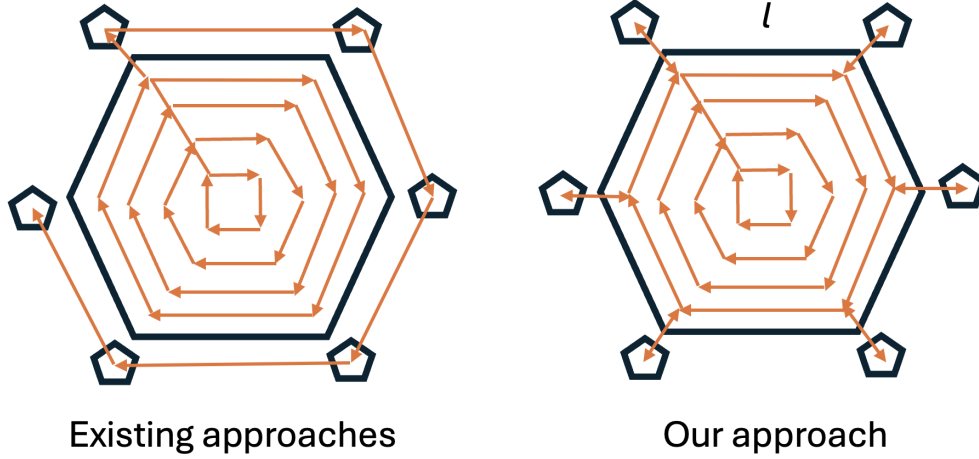


Figure 2: Comparison of the proposed approach (left) and the two-stage approach (right) for a triangle with three satellite regions.

Illustrative example.. Consider an instance with one large regular polygon of n edges of length l (a single region) and n unit-size satellite regions placed externally at unit distance from each vertex of the polygon. Figure 2 shows the case $n = 3$. Assume that the cost of covering the polygon is the same under both approaches. In our approach, the drone can leave the polygon at each vertex, cover the adjacent satellite, and resume the polygon. The satellite-covering cost is then $4n$ (unit inward detour, outward to satellite, back, unit back to polygon). In the two-stage approach, the satellite regions must be covered by separate trips from the depot; the best inter-region travel has cost at least $(n - 1)l$, where l is the polygon edge length. The ratio of the two cost gaps is then $(n - 1)l/(4n)$, which grows with l . The worst case for the two-stage gap among regular polygons is the triangle ($n = 3$), giving a ratio of $l/6$.

It should be noted that our approach does not *force* the drone to leave a region before completing it; it merely *allows* this whenever doing so reduces the total cost.

Complexity.. The problem is NP-hard: even with fixed chains, fixed entry/exit positions, and a single altitude, sequencing the inter-region legs under the endurance budget contains the travelling salesman problem (one region per operation with unit endurance) and the capacitated vehicle routing problem (packing multiple regions per operation) as special cases. This justifies both the exact MIQP approach for small instances and the heuristic of Section 5 for larger ones.

4. Mathematical Formulation

This section presents a mixed-integer quadratic programming formulation for the problem outlined in Section 3. The model has four families of constraints: *(i)* location constraints, which determine the placement of the entry/exit points along the polygonal chain of each region; *(ii)* drone path constraints, which govern the sequence of operations and eliminate subtours; *(iii)* distance constraints, which model the movements between points; and *(iv)* consumption constraints, which translate distances into energy and enforce the endurance budget. The decision variables used in the formulation are summarized in Tables 2 and 3.

Table 1: Main notation. Decision variables are detailed in Tables 2–3; constraint families Loc-C, Path-C, Dist-C, Ener-C, Dep-C, End-C, VI-C, S-C and W-C are defined in Sections 4–4.10.

Symbol	Meaning
<i>Sets and indices</i>	
$\mathcal{R}, R = \mathcal{R} $	regions and their number; r, r' index regions
$\mathcal{T}_r, \mathcal{S}_t$	candidate chains of r (t index) and segments of t (s index)
$\mathcal{I}_r$	candidate entry/exit points of r ($2R$ per region)
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	flight graph; v_0 depot, $\mathcal{V}'$ region vertices
$\mathcal{E}^{int}, \mathcal{E}^{ext}$	intra-region (chain) edges and inter-region legs
$\mathcal{O}$	operations (depot tours); o index
<i>Parameters</i>	
$h_t, \rho_h, \vec{v}_h^w$	chain altitude, air density, wind velocity at height h
$L(s), Q^s$	length and endpoints of chain segment s
$W^{\max}$	endurance (energy) budget per operation
<i>Key decision variables</i>	
α_t	1 if chain t is selected
$x_{vv'}^o, y_v^o, \zeta^o$	traversal, visitation and idle-operation indicators
$\lambda_v, \gamma_v, \mu_v^{ts}$	chain position, in-segment offset, segment assignment
$d_{uv}, w_{uv}, \ell_{uv}^o$	distance, energy and per-operation linearized cost

Table 1 collects the notation used throughout the model. Constraint families are labelled by block: Loc-C (location), Path-C (routing), Dist-C (distances), Ener-C (energy), Dep-C (depot legs), End-C (endurance), VI-C (valid inequalities), S-C (split points) and W-C (wind-aware energy); primed variants (e.g. Path-C'₄) denote the Edges-Approach counterparts introduced in Section 4.8.

Table 2: Binary and integer variables

Name	Domain	Range	Description
μ_v^{ts}	$\mathcal{V}' \times \{(t, s) : t \in \mathcal{T}_r, s \in \mathcal{S}_t\}$	$\{0, 1\}$	1, if vertex $v = (r, i)$ is located at segment s of chain t .
α_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$\{0, 1\}$	1, if polygonal chain t is selected.
$x_{vv'}^o$	$\mathcal{E} \times \mathcal{O}$	$\{0, 1\}$	1, if the drone traverses edge (v, v') in operation o .
y_v^o	$\mathcal{V}' \times \mathcal{O}$	$\{0, 1\}$	1, if vertex v is visited during operation o .
ζ^o	$\mathcal{O}$	$\{0, 1\}$	1, if all vertices are visited before starting operation o .
k^o	$\mathcal{O}$	$\{1, \dots, \mathcal{V}' \}$	number of vertices visited in operation o .
$\varphi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the drone moves from r to r' at the height of r .
$\psi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the height of r is lower than that of r' .

4.1. Location constraints

Idea. The entry/exit vertices are not fixed waypoints: each vertex $v = (r, i)$ floats along the coverage chains of its region, and the model must decide (a) *which* chain (altitude) it lies on and (b) *where* on that chain it sits. We encode (a) with a one-hot chain selector α_t and a segment assignment μ_v^{ts} , and (b) with a continuous in-segment offset $\gamma_v \in [0, 1]$; a single curvilinear coordinate λ_v then orders all vertices along the chain (Fig. 3). The constraints

Table 3: Continuous variables

Name	Domain	Range	Description
P_v	$\mathcal{V}'$	$\mathbb{R}^3$	Coordinates of the point associated to vertex v .
γ_v	$\mathcal{V}'$	$[0, 1]$	Position of vertex v within its selected segment.
λ_v	$\mathcal{V}'$	$[0, S_t] : t \in \mathcal{T}_r$	Global parametrization of vertex v on the selected chain.
η_v^{ts}	$\mathcal{V}' \times \{(t, s) : t \in \mathcal{T}_r, s \in \mathcal{S}_t\}$	$[0, 1]$	Linearization of $\gamma_v \cdot \mu_v^{ts}$.
d_{vs}^f	$\mathcal{V}'$	$[0, L_{\max}]$	Cumulative distance from the start of the chain to vertex v .
e_{vs}^f	$\mathcal{V}'$	$[0, W_{\max}]$	Cumulative energy from the start of the chain to vertex v .
$d_{vv'}$	$\mathcal{E}$	$\mathbb{R}^+$	Distance between vertex v and vertex v' .
$d_{vv'}^{xy}$	$\mathcal{E}^{ext}$	$\mathbb{R}^+$	Horizontal distance between v and v' .
$d_{vv'}^z$	$\mathcal{E}^{ext}$	$\mathbb{R}^+$	Vertical distance between v and v' .
$w_{vv'}$	$\mathcal{E}$	$\mathbb{R}^+$	Energy consumption between v and v' .
u_v^o	$\mathcal{V}' \times \mathcal{O}$	$[1, \mathcal{V}']$	Position of v in the tour of operation o (MTZ).
ℓ_{uv}^o	$\mathcal{E} \times \mathcal{O}$	$[0, \infty)$	Linearization of $x_{uv}^o \cdot d_{uv}$ (objective) and $x_{uv}^o \cdot w_{uv}$ (endurance).
ρ_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$[0, \rho(0)]$	Air density at the altitude of chain t (pre-computed from h_t).

below build this positioning in three steps: coordinates from assignment (Loc-C₁), one chain per region (Loc-C₂–C₃), and the λ ordering with endpoint fixing (Loc-C₄–C₈).

For each vertex $v = (r, i) \in \mathcal{V}'$ we define its spatial coordinates $P_v = (P_v^x, P_v^y, P_v^z) \in \mathbb{R}^3$. Let μ_v^{ts} be a binary variable that is one if vertex v is placed on segment s of chain t , and let $\gamma_v \in [0, 1]$ denote the relative position within that segment. The product $\gamma_v \cdot \mu_v^{ts}$ is linearized via the auxiliary continuous variable η_v^{ts} , constrained by standard Fortet inequalities ($0 \leq \eta_v^{ts} \leq \mu_v^{ts}$, $\gamma_v - (1 - \mu_v^{ts}) \leq \eta_v^{ts} \leq \gamma_v$). The coordinates of v are then given by:

$$P_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} [\mu_v^{ts} Q^s + \eta_v^{ts} (Q^{s+1} - Q^s)], \quad \forall v = (r, i) \in \mathcal{V}', \quad (\text{Loc-C}_1)$$

$$P_v^z = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} \mu_v^{ts} h_t, \quad \forall v = (r, i) \in \mathcal{V}', \quad (\text{Loc-C}_1^z)$$

where h_t is the altitude of chain t . Constraint (Loc-C₁^z) defines the vertical coordinate independently, since each chain t is associated with a single flight altitude. The segment assignment and chain selection are enforced by:

$$\sum_{s \in \mathcal{S}_t} \mu_v^{ts} = \alpha_t, \quad \forall v = (r, i) \in \mathcal{V}', \forall t \in \mathcal{T}_r, \quad (\text{Loc-C}_2)$$

$$\sum_{t \in \mathcal{T}_r} \alpha_t = 1, \quad \forall r \in \mathcal{R}, \quad (\text{Loc-C}_3)$$

where α_t is a binary variable that is one if chain t is selected. The parametrization of vertices along the chain is defined via the continuous variable $\lambda_v \in [0, S_t]$:

$$\lambda_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} (s - 1) \mu_v^{ts} + \gamma_v, \quad \forall v \in \mathcal{V}', \quad (\text{Loc-C}_4)$$

$$\lambda_v = 0, \quad \forall v = (r, 1), (r, 2), \quad (\text{Loc-C}_5)$$

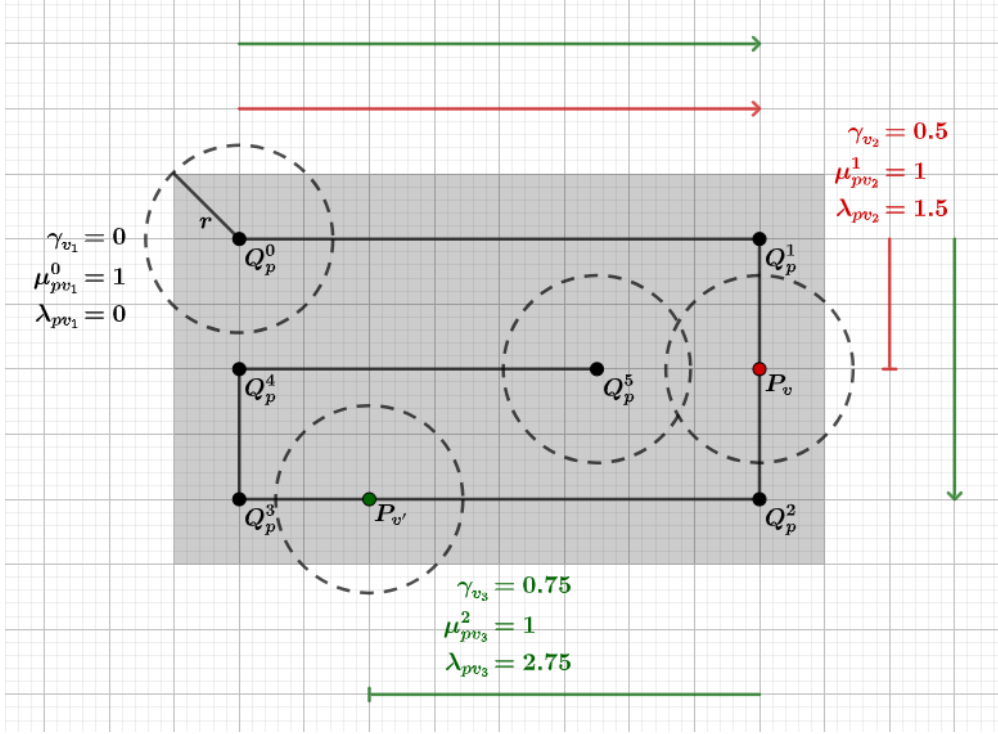


Figure 3: Parameterization of vertices

with the following fixing at the chain endpoints (start vertices $(r, 1), (r, 2)$ and end vertices $(r, 2R + 1), (r, 2R + 2)$):

$$\begin{aligned}
 \mu_v^{t0} &= \alpha_t, & \forall v = (r, 1), (r, 2), \forall t \in \mathcal{T}_r, & \quad (\text{Loc-C}_{5b}) \\
 \lambda_u &\leq \lambda_v, & \forall (u, v) \in \mathcal{E}_{RL}^{int} \cup \mathcal{E}_{RL}^S \cup \mathcal{E}_{RL}^T, & \quad (\text{Loc-C}_6) \\
 \lambda_u &= \lambda_v, & \forall (u, v) \in \mathcal{E}_{LR}^{int} \cup \mathcal{E}_{LR}^S \cup \mathcal{E}_{LR}^T, & \quad (\text{Loc-C}_7) \\
 \lambda_v &= \sum_{t \in \mathcal{T}_r} S_t \alpha_t, & \forall v = (r, 2R + 1), (r, 2R + 2), & \quad (\text{Loc-C}_8) \\
 \mu_v^{t, S_t-1} &= \alpha_t, & \forall v = (r, 2R + 1), (r, 2R + 2), \forall t \in \mathcal{T}_r. & \quad (\text{Loc-C}_{8b})
 \end{aligned}$$

Constraint (Loc-C₄) defines λ_v as the cumulative length from the chain start to the point, expressed as the index of the selected segment (minus one) plus the offset γ_v within that segment. Constraint (Loc-C₅) fixes the parametrization at zero for the start vertices. Constraint (Loc-C_{5b}) fixes the first segment for start vertices. Constraints (Loc-C₆) and (Loc-C₇) enforce the traversal order and the coincidence of launch–retrieve pairs, respectively. Constraint (Loc-C₈) sets λ to the total chain length for the end vertices, and (Loc-C_{8b}) fixes the last segment for those vertices. Together, Loc-C₁–C₈ pin every vertex to exactly one point of exactly one chain; the routing layer that follows can therefore treat the positions P_v as variables fully determined by the assignment decisions.

4.2. Drone path constraints

Idea. Once vertices have positions, the drone must string them into depot tours: each operation o is a closed walk starting and ending at v_0 that visits a subset of vertices, every vertex is visited exactly once across all operations, and no tour contains a cycle detached from the depot. We model this with traversal variables $x_{vv'}^o$ (which legs are flown in which operation) and visitation variables y_v^o derived from them; flow conservation (Path-C₂) plus depot departure/arrival (Path-C₁, Path-C₃) shape each operation, while Miller–Tucker–Zemlin ordering variables u_v^o (Path-C₇) rule out subtours. Operations are allowed to be idle once everything is covered, via the flag ζ^o .

In this subsection, we describe the set of constraints related to the operations that the drone must perform to visit all vertices described in $\mathcal{V}'$. In this context, we define several families of variables that model the path. Let $x_{vv'}^o$ be a binary variable that is one if the drone goes from vertex v to vertex v' in operation o . Let y_v^o be a binary variable that is one if the vertex v is visited during the operation o . Let also ζ^o be a binary variable that is one if all vertices are visited before starting the operation o . We count the number of vertices visited in operation o by means of the integer variable k^o . Additionally, u_v^o is a continuous variable that represents the position of vertex v in the tour of operation o , used in the subtour elimination constraints.

$$\begin{aligned}
\sum_{v' \in \mathcal{V}} x_{v_0 v'}^o &= 1 - \zeta^o, & \forall o \in \mathcal{O}, & \quad (\text{Path-C}_1) \\
\sum_{v' \in \mathcal{V}} x_{v' v}^o - \sum_{v' \in \mathcal{V}} x_{vv'}^o &= 0, & \forall v \in \mathcal{V}', \forall o \in \mathcal{O}, & \quad (\text{Path-C}_2) \\
\sum_{v' \in \mathcal{V}} x_{v' v_0}^o &= 1 - \zeta^o, & \forall o \in \mathcal{O}, & \quad (\text{Path-C}_3) \\
\sum_{v' \in \mathcal{V}} x_{vv'}^o &= y_v^o, & \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, & \quad (\text{Path-C}_4) \\
\sum_{v' \in \mathcal{V}} x_{v' v}^o &= y_v^o, & \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, & \quad (\text{Path-C}_5) \\
\sum_{o \in \mathcal{O}} y_v^o &= 1, & \forall v \in \mathcal{V}', & \quad (\text{Path-C}_6) \\
u_v^o - u_{v'}^o + 1 &\leq |\mathcal{V}'|(1 - x_{vv'}^o), & \forall v, v' \in \mathcal{V}' : v \neq v', \forall o \in \mathcal{O}, & \quad (\text{Path-C}_7) \\
1 &\leq u_v^o \leq |\mathcal{V}'|, & \forall v \in \mathcal{V}', \forall o \in \mathcal{O}. & \quad (\text{Path-C}_{7b}) \\
\sum_{o \in \mathcal{O}} x_{vv'}^o &= 1, & \forall v, v' \in \mathcal{V}' : (v, v') \in \mathcal{E}_{RL}^{int} \cup \mathcal{E}_{RL}^S \cup \mathcal{E}_{RL}^T. & \quad (\text{Path-C}_8)
\end{aligned}$$

Constraints (Path-C₁) ensure that the drone exits from the depot if there is a vertex that is not already visited in operation o . Constraints (Path-C₂) are the flow conservation constraints. Constraints (Path-C₃) guarantee that drone operation ends in the depot if there is a vertex that has not been visited in operation o . Constraints (Path-C₄) and (Path-C₅) ensure that the vertex v is visited if there is a path passing through this vertex. Constraints (Path-C₆) impose that every vertex must be visited in some operation. Inequalities

(Path-C₇) are the Miller–Tucker–Zemlin (MTZ) subtour elimination constraints [18], which use the auxiliary variables u_v^o to enforce that the tours in each operation contain no cycles that exclude the depot. The bounds (Path-C_{7b}) define the range of these continuous variables. Finally, constraints (Path-C₈) guarantee that each polygonal chain is completely traversed by requiring that all edges of the path graph associated with each region are traversed in some operation.

Valid Inequalities

Idea. The path constraints above admit many symmetric solutions (e.g. permuting idle operations) that slow down branch-and-bound without changing the optimum. The inequalities VI-C₁–C₄ cut off such fractional and symmetric solutions: they force idle operations to come last, count how many vertices each operation covers, and tie that count to the idle flag so the relaxation “sees” that an active operation must do useful work.

In this subsection, we present a set of valid inequalities for the representation of the drone path that are included in the formulation to enhance it. Let ζ^o be a binary variable that takes the value of one if all vertices have been visited before starting the operation o , and zero otherwise. If this is the case, then all vertices must also be visited before the operation $o + 1$. Therefore, the ζ variables must satisfy the following monotonicity constraint:

$$\zeta^o \leq \zeta^{o+1}, \quad \forall o \in \mathcal{O} \setminus \{O\}. \quad (\text{VI-C}_1)$$

We define k^o as the number of vertices visited during the operation o , excluding the depot. The variables y can be used to determine this quantity, where we recall that y_v^o equals one if the vertex v is visited in operation o . Thus:

$$k^o = \sum_{v \in \mathcal{V}'} y_v^o, \quad \forall o \in \mathcal{O}. \quad (\text{VI-C}_2)$$

If ζ^o is equal to one, it implies that all vertices in $\mathcal{V}'$ must have been visited in previous operations. This relationship is enforced through the following inequality:

$$\sum_{o'=1}^{o-1} k^{o'} \geq |\mathcal{V}'| \zeta^o, \quad \forall o \in \mathcal{O}. \quad (\text{VI-C}_3)$$

To refine the feasible solution space, we assume, without loss of generality, that an operation o cannot consist in remaining at the depot, if there are still targets left to visit. This requirement is ensured by the following constraint:

$$k^o \geq 1 - \zeta^o, \quad \forall o \in \mathcal{O}. \quad (\text{VI-C}_4)$$

4.3. Distance constraints

Idea. Routing needs leg lengths, but vertex positions are variables, so distances cannot be precomputed—except along a chain, where the λ ordering makes them trivial. The trick is therefore twofold: on intra-region edges read the length off an accumulated-distance variable $df s_v$ (a difference of two such variables, Dist-C₁–C₂), and on inter-region legs introduce explicit distance variables bounded below by the Euclidean norm via a second-order

cone (Dist-C₃–C₆). Since the objective minimizes total length, each “ $\geq$ ” bound is tight at optimality.

Since vertex positions are decision variables, all distances must be expressed through continuous variables. We first consider pairs of vertices belonging to the same region, then pairs in different regions.

Intra-region distances.. For each region r , the polygonal chain t is composed of segments $\mathcal{S}_t$ of known lengths $L(s) = \|Q^{s+1} - Q^s\|_2$. Rather than introducing bilinear products $\mu_v^s \mu_{v'}^{s'}$, we define an auxiliary continuous variable $df s_v$ that accumulates the distance from the start of the chain to vertex v :

$$df s_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} [\text{cum_before}(s) \mu_v^{ts} + L(s) \eta_v^{ts}], \quad \forall v \in \mathcal{V}', \quad (\text{Dist-C}_1)$$

$$d_{uv} = df s_v - df s_u, \quad \forall (u, v) \in \mathcal{E}_{RL}^{int}, \quad (\text{Dist-C}_2)$$

$$d_{uv} = 0, \quad \forall (u, v) \in \mathcal{E}_{LR}^{int}, \quad (\text{Dist-C}_3)$$

where $\text{cum_before}(s) = \sum_{s' < s} L(s')$ is the length of the chain up to the start of segment s , and $\eta_v^{ts} = \gamma_v \cdot \mu_v^{ts}$ is the same linearization variable used in (Loc-C₁). For RL edges the distance is simply the difference of cumulative distances from the chain start; for LR edges the two vertices coincide ($\lambda_u = \lambda_v$ from (Loc-C₇)) so the distance is zero.

Inter-region distances.. For vertices in different regions we decompose the movement into horizontal and vertical components. Let $d_{vv'}^{xy}$ and $d_{vv'}^z$ be continuous variables representing the horizontal and vertical distances, respectively. The total distance is then:

$$d_{vv'} = d_{vv'}^{xy} + d_{vv'}^z, \quad \forall (v, v') \in \mathcal{E}^{ext}, \quad (\text{Dist-C}_4)$$

$$d_{vv'}^{xy} \geq \|(P_v^x, P_v^y) - (P_{v'}^x, P_{v'}^y)\|_2, \quad \forall (v, v') \in \mathcal{E}^{ext}, \quad (\text{Dist-C}_5)$$

$$d_{vv'}^z \geq P_v^z - P_{v'}^z, \quad \forall (v, v') \in \mathcal{E}^{ext}, \quad (\text{Dist-C}_6)$$

$$d_{vv'}^z \geq P_{v'}^z - P_v^z, \quad \forall (v, v') \in \mathcal{E}^{ext}. \quad (\text{Dist-C}_7)$$

Constraint (Dist-C₅) is a second-order cone constraint that computes the Euclidean horizontal distance. Constraints (Dist-C₆)–(Dist-C₇) give the absolute vertical difference via a standard linearization. At this point every edge $(u, v) \in \mathcal{E}$ carries a well-defined length d_{uv} : exact along chains, Euclidean across regions. The next block converts these lengths into energy.

4.4. Consumption constraints

The energy consumed by the drone is modelled using the drag formula $w = \frac{1}{2} A c \rho \nu d$, where A is the frontal area, c the drag coefficient, ρ the air density, ν the drone speed, and d the travelled distance. We define the constants $E^{xy} = E^z = \frac{1}{2} c A$ for brevity. *Units:* throughout Sections 4–4.9 energy is measured in the model’s nominal units (drag constants at cruise speed, calm air), which we write as pseudo-joules to distinguish them from the real joules of the wind-aware extension in Section 4.10. As with distances, we treat intra-region and inter-region pairs separately.

Intra-region energy... Following the same approach as for distances, we introduce an auxiliary continuous variable efs_v that accumulates the energy from the start of the chain to vertex v :

$$efs_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} C_{ts} [\text{cum_before}(s) \mu_v^{ts} + L(s) \eta_v^{ts}], \quad \forall v \in \mathcal{V}', \quad (\text{Ener-C}_1)$$

$$w_{uv} = efs_v - efs_u, \quad \forall (u, v) \in \mathcal{E}_{RL}^{int}, \quad (\text{Ener-C}_2)$$

$$w_{uv} = 0, \quad \forall (u, v) \in \mathcal{E}_{LR}^{int}, \quad (\text{Ener-C}_3)$$

where $C_{ts} = E^{xy} \rho_t \nu_{ts}^d$ is the energy per unit distance on segment s of chain t , computed as a constant during pre-processing. The drone ground speed ν_{ts}^d along segment s is obtained from $\vec{\nu}_{ts}^d = v_{\text{cruise}} \frac{Q^{s+1} - Q^s}{\|Q^{s+1} - Q^s\|} - \vec{\nu}_h^w$ for forward traversal (and the opposite sign for backward traversal).

Inter-region energy... *Idea.* Flying from region r to region r' burns horizontal energy at the air density of the departure altitude and vertical energy only for the climbed part. Both depend on decisions: the departure chain (via α_t) selects the density, and the relative heights of r and r' decide the climb direction. Two auxiliary binaries disentangle this: $\varphi_{rr'}$ is one if the drone departs from region r at the altitude selected for that region, and $\psi_{rr'}$ is one if the altitude of r is lower than that of r' . Each branch of (Ener-C₄) below is then a product of binaries with distances, linearized exactly with Fortet envelopes. The energy consumption is then:

$$\begin{aligned} w_{vv'} = & \varphi_{rr'} \left(E^{xy} d_{vv'}^{xy} \sum_{t \in \mathcal{T}_r} \alpha_t \rho_t |\vec{\nu}_{rr't}^d| + E^z \psi_{rr'} d_{vv'}^z \frac{1}{2} \sum_{\tau \in \{r, r'\}} \sum_{t \in \mathcal{T}_\tau} \alpha_t \rho_t \nu^z \right) \\ & + (1 - \varphi_{rr'}) \left(E^{xy} d_{vv'}^{xy} \sum_{t \in \mathcal{T}_{r'}} \alpha_t \rho_t |\vec{\nu}_{rr't}^d| + E^z (1 - \psi_{rr'}) d_{vv'}^z \frac{1}{2} \sum_{\tau \in \{r, r'\}} \sum_{t \in \mathcal{T}_\tau} \alpha_t \rho_t \nu^z \right), \end{aligned} \quad (\text{Ener-C}_4)$$

for all $(v, v') \in \mathcal{E}^{ext}$ with $v = (r, i), v' = (r', i')$. All bilinear products involving $\varphi_{rr'}$, $\psi_{rr'}$, and α_t are linearized explicitly using standard Fortet constraints.

Depot energy... For movements to and from the depot $v_0 = (0, 0, 0)$, we use a simplified model with constant sea-level air density ρ_0 and a constant horizontal cruise speed ν^{xy} . The depot-leg distance is decomposed as:

$$d_{v_0v} = d_v^{xy} + d_v^z, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_1)$$

$$d_v^{xy} \geq \|(P_v^x, P_v^y)\|_2, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_2)$$

$$d_v^z \geq |P_v^z|, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_3)$$

$$w_{v_0v} = E^{xy} \rho_0 \nu^{xy} d_v^{xy} + \frac{1}{2} E^z \rho_0 \nu^z d_v^z, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_4)$$

$$w_{vv_0} = E^{xy} \rho_0 \nu^{xy} d^{xy}, \quad \forall v \in \mathcal{V}'. \quad (\text{Dep-C}_5)$$

The ascent energy (Dep-C₄) includes both horizontal and vertical work; the descent (Dep-C₅) assumes the drone recovers no energy (vertical contribution is zero).

4.5. Endurance constraint

Idea. An operation is feasible only if the energy of the legs it actually flies fits the battery. That is a product of two variables (fly the leg $\times$ leg cost), so we introduce one auxiliary ℓ_{uv}^o per leg–operation pair that equals w_{uv} when $x_{uv}^o = 1$ and 0 otherwise (big-M, End-C₁–C₄); the budget $W^{\max}$ then caps the sum per operation (End-C₅). The same device serves the objective, which needs $x_{uv}^o \cdot d_{uv}$.

The bilinear product $x_{uv}^o w_{uv}$ in the endurance constraint (and $x_{uv}^o d_{uv}$ in the objective) is linearized via auxiliary continuous variables ℓ_{uv}^o with the following big-M formulation:

$$\ell_{uv}^o \leq x_{uv}^o \cdot M, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-C}_1)$$

$$\ell_{uv}^o \leq w_{uv}, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-C}_2)$$

$$\ell_{uv}^o \geq w_{uv} - (1 - x_{uv}^o)M, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-C}_3)$$

$$\ell_{uv}^o \geq 0, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-C}_4)$$

where M is a sufficiently large constant. The endurance of each operation is then enforced by:

$$\sum_{(u,v) \in \mathcal{E}} \ell_{uv}^o \leq W^{\max}, \quad \forall o \in \mathcal{O}. \quad (\text{End-C}_5)$$

4.6. Objective function

The objective is to minimize the total distance travelled across all operations, equivalently the total flight time since the drone speed is constant per segment. Using the same linearization as above,

$$\min W = \sum_{(u,v) \in \mathcal{E}} \sum_{o \in \mathcal{O}} \ell_{uv}^o, \quad (\text{Objective})$$

where ℓ_{uv}^o now linearizes $x_{uv}^o \cdot d_{uv}$ (the same variables can be reused because a single ℓ_{uv}^o per edge–operation pair suffices, and each ℓ_{uv}^o is constrained by both d_{uv} and w_{uv} via the big-M formulation above). Minimizing distance is equivalent to minimizing flight time at constant cruise speed, so W is both the tour length and (up to constants) the mission duration.

4.7. Formulation

The complete formulation of the CPP follows, grouping the constraints by family:

$$\text{minimize } W = \sum_{(u,v) \in \mathcal{E}} \sum_{o \in \mathcal{O}} \ell_{uv}^o \quad (\text{Formulation})$$

$$\text{subject to } (\text{Path-C}_1) - (\text{Path-C}_{7b}),$$

$$\begin{aligned}
&(\text{VI-C}_1), (\text{VI-C}_2), (\text{VI-C}_3) - (\text{VI-C}_4), \\
&(\text{Loc-C}_1) - (\text{Loc-C}_{8b}), \\
&(\text{Dist-C}_1), (\text{Dist-C}_2) - (\text{Dist-C}_7), \\
&(\text{Ener-C}_1), (\text{Ener-C}_2) - (\text{Ener-C}_4), (\text{Dep-C}_1) - (\text{Dep-C}_5), \\
&(\text{End-C}_1) - (\text{End-C}_5)
\end{aligned}$$

4.8. Alternative Edges-Approach formulation

Idea. The Vertex-Approach tracks every vertex visit explicitly (y_v^o) and forbids subtours with MTZ ordering variables. When regions grow, vertex pairs explode—yet covering a ring only requires *traversing* its edges, not naming each visit. The Edges-Approach therefore drops y_v^o and the MTZ variables entirely, counts coverage directly on the traversal variables x_{uv}^o (a rural-postman perspective), and forbids subtours with DFJ cuts separated lazily only when violated. Location, distance, energy and endurance blocks are untouched; only the routing layer changes, and its constraints mirror Path-C/VI-C one by one (primed tags).

Table 4: Binary and integer variables for the Edges-Approach formulation

Name	Domain	Range	Description
μ_v^s	$\mathcal{V}' \times \{s \in \mathcal{S}_t : t \in \mathcal{T}_r\}$	$\{0, 1\}$	1, if vertex $v = (r, i)$ is located at the segment $s \in \mathcal{S}_t$, where $t \in \mathcal{T}_r$.
α_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$\{0, 1\}$	1, if polygonal chain t is selected.
$x_{vv'}^o$	$\mathcal{E} \times \mathcal{O}$	$\{0, 1\}$	1, if the drone traverses the edge (v, v') in operation o .
ζ^o	$\mathcal{O}$	$\{0, 1\}$	1, if all vertices are visited before starting operation o .
k^o	$\mathcal{O}$	$[1, 2, \dots, \mathcal{E}^{int}]$	number of intra-ring edges covered in operation o .
$\varphi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the drone moves from region r to region r' at the height selected to visit region r .
$\psi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the height selected to visit region r is lower than the height selected to visit region r' .

The formulation presented above treats each vertex in $\mathcal{V}'$ individually: location variables μ_v^s , γ_v are defined per vertex, and path variables $x_{vv'}^o$ are defined per pair of vertices. When the number of regions grows, the combinatorial explosion of vertex pairs—especially in $\mathcal{E}^{ext}$ —can make the model difficult to solve. An alternative representation can be obtained by eliminating the per-vertex visitation variables y_v^o entirely and using the traversal variables x_{uv}^o directly to count covered edges, following a perspective akin to the rural postman problem.

In this alternative formulation, the location constraints (Loc-C₁ through Loc-C₅, ρ -selection) and the distance and energy constraints (Dist-C, Ener-C, End-C) remain unchanged, as they do not depend on y_v^o . The binary and integer variables for this formulation are listed in Table 4; the continuous variables are the same as in Table 3 of the Vertex-Approach formulation, except for the MTZ auxiliary variables u_v^o which are not needed (subtour elimination is enforced via DFJ lazy cuts). The drone path constraints are modified as follows.

Drone path constraints (Edges-Approach). The flow conservation (Path-C₂) and the depot constraints (Path-C₁, Path-C₃) are kept unchanged. The MTZ subtour elimination (Path-C₇), which relies on auxiliary vertex-position variables u_v^o , is replaced by Dantzig–Fulkerson–Johnson (DFJ) exponential subtour elimination constraints separated dynamically via a lazy constraint callback during the branch-and-cut search. At each feasible integer incumbent, a breadth-first search from the depot identifies, for each operation o , the set of vertices

reachable from the depot. If a proper subset $S \subsetneq \mathcal{V}'$ that does not contain the depot forms a connected component, the following cut is added to the model:

$$\sum_{u \in S, v \notin S} x_{uv}^o \geq 1, \quad \forall S \subsetneq \mathcal{V}' : v_0 \notin S, \forall o \in \mathcal{O}. \quad (\text{DFJ-C}_1)$$

This constraint enforces that at least one edge must leave S , thereby preventing subtours that avoid the depot. The lazy callback avoids adding the full exponential family of DFJ constraints a priori and integrates naturally with the branch-and-cut framework of modern solvers such as Gurobi.

The degree constraints are relaxed: instead of coupling each traversal to a vertex-visitation variable, we bound the number of traversals incident to each vertex:

$$\sum_{v' \in \mathcal{V}} x_{vv'}^o \leq 1, \quad \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, \quad (\text{Path-C}'_4)$$

$$\sum_{v' \in \mathcal{V}} x_{v'v}^o \leq 1, \quad \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, \quad (\text{Path-C}'_5)$$

The constraint Path-C₈ (each intra edge traversed exactly once) is kept unchanged. Since intra-ring edges are directed (only the forward direction exists), the traversal variable x_{uv}^o already serves as a coverage indicator for the edge (u, v) , and a separate coverage variable is unnecessary.

For idle operations ($\zeta^o = 1$), all traversal variables are forced to zero via the constraint

$$\sum_{(u,v) \in \mathcal{E}} x_{uv}^o \leq (|\mathcal{V}'|+1)(1 - \zeta^o), \quad \forall o \in \mathcal{O}, \quad (\text{Path-C}_9)$$

where the big-M coefficient $|\mathcal{V}'|+1$ is a tight upper bound on the number of edges in any feasible tour (at most one outgoing edge per vertex plus the depot departure). This is significantly tighter than the original bound $|\mathcal{V}|^2$.

Adapted valid inequalities.. The monotonicity constraint (VI-C₁) (VI-C₁) and VI-C₄ (VI-C₄) are unchanged. The counting constraint VI-C₂ and VI-C₃ are updated to use the Edges-Approach variables:

$$k^o = \sum_{(u,v) \in \mathcal{E}^{int}} x_{uv}^o, \quad \forall o \in \mathcal{O}, \quad (\text{VI-C}'_2)$$

$$\sum_{o'=1}^{o-1} k^{o'} \geq |\mathcal{E}^{int}| \zeta^o, \quad \forall o \in \mathcal{O}. \quad (\text{VI-C}'_3)$$

Equivalence.. The traversal variables x_{uv}^o replace the vertex-visitation variables y_v^o in the valid inequalities: k^o counts edges traversed rather than vertices visited. In the Vertex-Approach formulation, the degree constraints Path-C₄–C₅ couple each vertex visit to two incident edges, so each intra-ring edge contributes exactly one to k^o via its two endpoint

vertices. Here it contributes directly via x_{uv}^o , and Path-C₈ ensures each edge is traversed exactly once. The two formulations share the same set of feasible integer solutions and therefore the same optimal value. The Edges-Approach formulation is implemented alongside the Vertex-Approach one in our computational framework, allowing a direct comparison of their performance. Both, however, share one structural restriction—each ring is covered as a single unbroken loop—which the next section removes.

4.9. Multiple split points per ring

Idea. A ring covered as one loop forces its entry and exit points to coincide, so a tour can never slip out mid-ring to visit a neighbour and come back. Allowing K arcs per ring (Fig. 4) turns each ring into a sequence of pieces joined by zero-cost connectors; pieces of the same ring may then live in different operations or be interleaved with other regions inside one operation. Section 4.9 walks through a concrete $K = 2$ solution arc by arc.

The Vertex-Approach formulation of Section 4 covers each ring as a single full loop: exactly one coincident launch/retrieve pair is placed per ring, so a ring can never be split across operations (nor entered and exited at different points of the same operation). We generalize the model with a parameter $K \geq 1$, the number of *continuous split points* allowed per ring; $K = 1$ reproduces the original behaviour.

For $K \geq 2$, each ring κ is partitioned into K arcs in series. With vertices (ℓ_j, r_j) , $j = 1, \dots, K$, denoting the launch/retrieve pair of arc j , the split positions are ordered along the ring by

$$\lambda_{\ell_1} = 0, \tag{S-C_1}$$

$$\lambda_{r_j} = \lambda_{\ell_{j+1}}, \quad j = 1, \dots, K-1, \tag{S-C_2}$$

$$\lambda_{r_K} = \sum_{t \in \mathcal{T}_r} S_t \alpha_t, \tag{S-C_3}$$

$$\lambda_{\ell_j} \leq \lambda_{r_j}, \quad j = 1, \dots, K, \tag{S-C_4}$$

where S_t is the number of segments of the ring on chain t (so (S-C₃) anchors the last retrieve point at the ring end of the selected chain). Zero-length arcs are allowed, hence K is an upper bound on the number of pieces. Each arc (ℓ_j, r_j) must be traversed exactly once in either direction (the analogue of Path-C₈), and consecutive arcs are linked inside an operation by zero-length *connector* edges (r_j, ℓ_{j+1}) and (ℓ_{j+1}, r_j) , plus the wrap connectors (r_K, ℓ_1) and (ℓ_1, r_K) joining the coincident ring endpoints. Arc lengths follow from the cumulative-distance variables as $d_{\ell_j r_j} = d f s_{r_j} - d f s_{\ell_j}$ (pseudo-energy), or exactly under wind with the segment-overlap indicators of Section 4.10.

Two structural facts are worth noting. First, anchoring one cut at $\lambda = 0$ is without loss of optimality up to a single extra arc: any tour whose ring is cut at m points is reproduced with $K = m + 1$ by adding the forced cut at 0 and covering the two resulting sub-arcs consecutively in the same operation at zero extra cost. Second, the models are *nested*: a K -arc solution maps onto $K + 1$ arcs by collapsing superfluous arcs to zero length, so optimal values are non-increasing in K . The practical question is how small K can be before the objective stops improving.

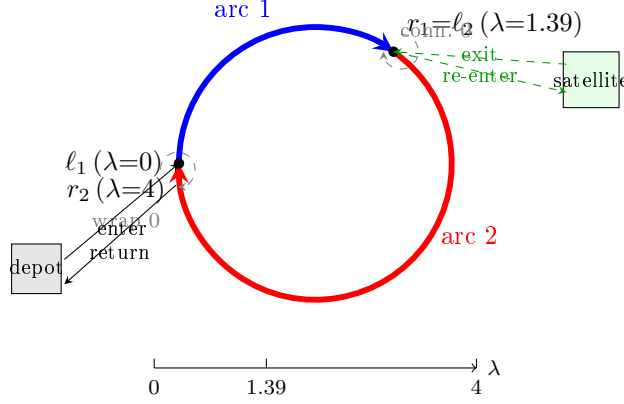


Figure 4: A ring split into $K = 2$ arcs (numbers from the demo solution of Fig. 5). The tour enters at ℓ_1 ($\lambda = 0$, S-C₁), covers arc 1, exits mid-ring at $r_1 = \ell_2$ ($\lambda = 1.39$, S-C₂) to visit the satellite, re-enters at the same point via the zero-length connector, covers arc 2, and returns to the depot from r_2 ($\lambda = 4$, S-C₃); S-C₄ keeps every arc non-empty in the λ order.

Table 5: Best objective found vs. number of split points K per ring. Showcase: pseudo-energy, $E = 150$ J ($K=1$ optimal). Demo and irregular: wind-aware real joules ($E = 173$ J and $E = 153$ J). Bold marks the best value per instance.

K	Showcase ($E = 150$)	Demo ($E = 173$)	Irregular ($E = 153$)
1	708.0	677.6	776.1
2	697.2	652.1	582.0
3	700.1	630.9	559.7
4	708.0	621.7	559.7
5	—	624.9	1312.0

Table 5 reports the best objective found for $K = 1, \dots, 5$ on three instances (a 3-region showcase in pseudo-energy and two wind-aware demos, see Section 4.10). The knee is consistently at $K = 2$ –3: one split point per ring already improves the single-loop optimum by 1.5–25% (showcase $708.0 \rightarrow 697.2$; demo $677.6 \rightarrow 652.1$; irregular $776.1 \rightarrow 581.9$), a second cut helps on the larger instances (demo $652.1 \rightarrow 630.9 \rightarrow 621.7$; irregular down to 559.7), and $K \geq 4$ adds nothing within solver reach while the model size explodes (Fig. 11b: 3k variables at $K = 1$ vs. 66k at $K = 5$ on the demo). Figure 5 shows the mechanism on the demo instance: with $K = 2$ the single operation enters and leaves the central region four times, interleaving satellite visits between ring arcs—impossible with coincident entry/exit points.

Worked example: the $K = 2$ demo tour

Table 6 walks through the single operation of Fig. 5 arc by arc (vertex indices decoded from the solver output; λ cuts verified against the solution file). The tour covers the satellite rings opportunistically between pieces of the central region R0: R0-ring1 is split across

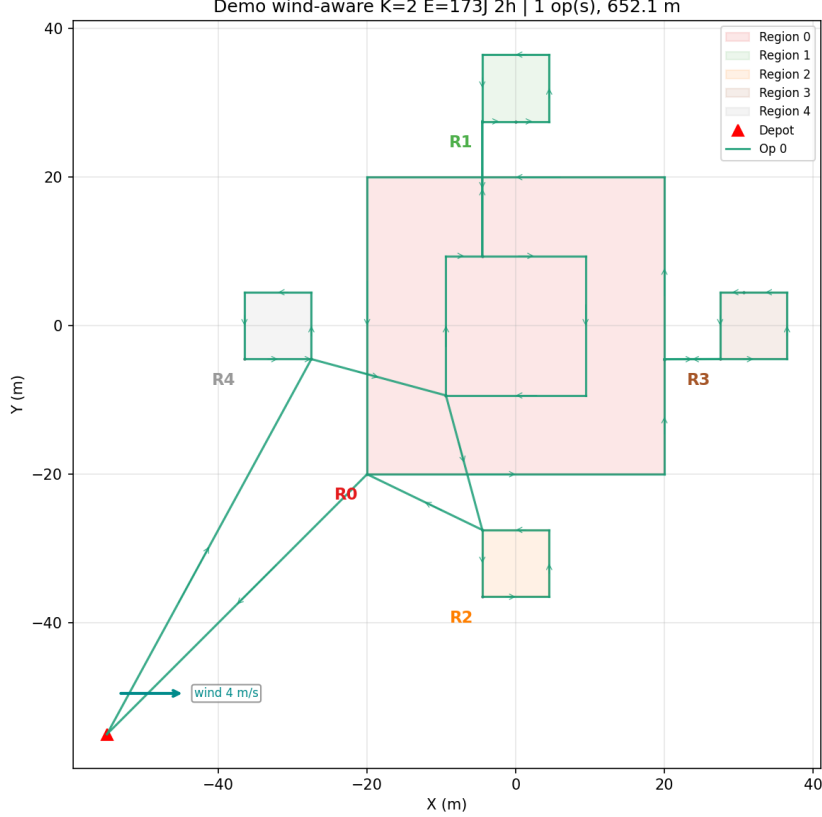


Figure 5: Wind-aware demo solution with $K = 2$ split points per ring ($E = 173\text{J}$, one operation, 652.1 m): the tour enters and leaves the central region four times (coloured arcs), covering ring pieces between satellite visits. Cut positions are fractional ($\lambda = 1.39, 1.26$ on the central rings; $0.5, 3.0, 2.65, 1.0$ on the satellites), i.e. strictly inside ring segments rather than at ring vertices.

steps 2 and 4, R0-ring0 across steps 6 and 8. Each re-entry reuses the cut point as both exit and entry ($r_1 = \ell_2$ in Fig. 4), so no distance is wasted. With $K = 1$ this tour is infeasible: each ring would have to be covered in one contiguous visit with coincident entry/exit, so R0 could not be interleaved with the satellites (the $K = 1$ optimum on this instance is 677.6 m — 25.5 m more, Table 5).

4.10. Wind-aware energy model

Idea. Drag depends on *airspeed*, not ground speed: holding a ground track against the wind costs more than riding with it. With the cruise speed of 15 m/s used throughout this paper, an 8 m/s east wind makes an eastbound leg cost only $7/15$ of its calm-air energy while the same leg westbound costs $23/15$ —a factor-three asymmetry the pseudo-energy misses entirely. The extension below prices every leg by its true airspeed: exact per-heading rates on ring segments ($W-C_1$), centroid-direction rates on transits and depot legs. Crucially, all of these rates are *constants* computed before the solve (headings are known geometry), so the model stays a MIQP.

The pseudo-energy used above (metres on coverage legs and transits) ignores the wind entirely, except through a crude factor on depot legs. We extend the model with real energy

Table 6: Arc-by-arc walk of the $K = 2$ demo tour (652.1 m, one operation). Cuts sit at fractional λ positions, e.g. R0-ring0 is split at $\lambda = 1.39$ of 4.0.

Step	Region	Arc(s) covered (λ interval)
1	R4 (satellite)	arc1 [1.0→4.0], then arc0 [0→1.0] (full ring)
2	R0 (centre), ring1	arc0 [0→1.26] — exit mid-ring
3	R1 (satellite)	arc0 [0→0.5], arc1 [0.5→4.0] (full ring)
4	R0, ring1	arc1 [1.26→4.0] — ring1 complete
5	R2 (satellite)	arc1 [3.0→4.0], then arc0 [0→3.0] (full ring)
6	R0, ring0	arc0 [0→1.39] — exit mid-ring
7	R3 (satellite)	arc0 [0→2.65], arc1 [2.65→4.0] (full ring)
8	R0, ring0	arc1 [1.39→4.0] — ring0 complete, return to depot

in joules, evaluated from the airspeed required to hold each ground track. The wind blows in a single constant direction $\vec{u}^w$ with power-law speed profile $\nu_h^w = \nu_{10}^w(h/10)^a$ (hellmann exponent $a = 0.2$); air density ρ_h follows the barometric formula. On a leg flown at ground velocity $\vec{v}$, the drone airspeed is $\vec{v}_h^d = \vec{v} - \vec{v}_h^w$ and the drag energy per metre is $E^{xy}\rho_h|\vec{v}_h^d|$ with $E^{xy} = \frac{1}{2}cA$.

Intra-ring arcs.. Ring segment s of chain t at height h_t has known unit tangent $\vec{t}_s$, hence a precomputable energy rate $C_{ts} = E^{xy}\rho_{h_t}|v_{\text{cruise}}\vec{t}_s - \vec{v}_{h_t}^w|$ (and analogously C_{ts}^b for backward traversal). A full loop then costs $\sum_s C_{ts}L(s)$ linearly in the chain-selection variables α_t . For a K -arc with $K \geq 2$, the energy of arc (u, v) is the exact segment-weighted length

$$w_{uv} = \sum_t \sum_s C_{ts} L(s) [I_{uvs}^t - \eta_u^{ts} - (\mu_v^{ts} - \eta_v^{ts})], \quad (\text{W-C}_1)$$

where the binary indicator I_{uvs}^t tells whether the arc covers segment s on chain t , enforced by $I_{uvs}^t \geq \sum_{s' \leq s} \mu_u^{ts'} + \sum_{s' \geq s} \mu_v^{ts'} - 1$ together with the two upper bounds $I_{uvs}^t \leq \sum_{s' \leq s} \mu_u^{ts'}$ and $I_{uvs}^t \leq \sum_{s' \geq s} \mu_v^{ts'}$; the η terms discount the uncovered fractional ends. The backward direction uses C_{ts}^b with mirrored partials. All other constraints, including distances (the objective stays a distance), are unchanged.

Inter-region and depot legs.. Following the paper’s centroid approximation, the ground direction of a leg (r, r') is fixed offline as the unit vector between region centroids, giving precomputable airspeed factors $F_{rr't} = E^{xy}\rho_{h_t}|v_{\text{cruise}}\vec{d}_{rr'} - \vec{v}_{h_t}^w|$ per departure chain t (plus a vertical term $V_t = \frac{1}{2}E^z\nu^z\rho_{h_t}$). The only remaining bilinearities are products of these constants with distances times the one-hot selection α_t (e.g. $d_{vv'}^{xy} \cdot \alpha_t$), linearised exactly with Fortet envelopes. Depot legs use the same construction with the fixed depot-to-centroid direction. The mode is opt-in (`wind_aware` flag, off by default) and orthogonal to K .

Switching to joules rescales endurance: on the showcase $E_{\min}$ drops from 153 pseudo-J to 68.7 J real. Table 7 validates the model at tight (104 J) and loose (207 J) budgets under strong wind (8 m/s east) versus calm air: optimal tours and objectives are identical (594.7 m in 2 ops; 500.2 m in 1 op). Wind therefore acts here as a second-order perturbation—it

Table 7: Wind-aware validation on the showcase ($E_{\min} = 68.7$ J real): strong wind (8 m/s east) vs. calm. Identical optimal tours and objectives: wind does not change the optimum here.

Case	E (J)	Obj. (m)	Ops	Gap (%)
tight wind8	104	594.7	2	1.9
tight calm	104	594.7	2	2.0
loose wind8	207	500.2	1	2.0
loose calm	207	500.2	1	2.0

reshuffles ≈ 8 –16% of leg energy at 8 m/s but does not change the optimal structure, which is driven by geometry and the endurance budget. The $K = 2$ wind-aware optimum (652.1 m, Fig. 5) confirms splits remain beneficial under real energy accounting.

5. Heuristic Solution Method

The exact mixed-integer formulation presented in Section 4 becomes computationally intractable on large-scale instances involving numerous regions or high-resolution rings. To address this limitation, we develop a dedicated multi-phase constructive heuristic. The algorithm is designed to rapidly generate high-quality feasible solutions in a fraction of a second, which can subsequently be supplied as a warm start to the exact branch-and-bound solver.

As summarized in Algorithm 1, the solution procedure is decomposed into three sequential phases:

1. **Pre-processing (§5.1.1–5.1.2):** Selects an appropriate flight chain (comprising altitude and coverage pattern) for each region and computes the geometric properties of the resulting surveillance rings.
2. **Construction (§5.2.1–5.2.2):** Establishes a global visit sequence (giant tour) over all rings and partitions it into endurance-feasible operations.
3. **Improvement (§5.3.1–5.3.2):** Refines entry points and applies metaheuristic local-search moves (intra-operation 2-opt and inter-operation Or-opt) to minimize total energy consumption.

5.1. Phase 1: Pre-processing

5.1.1. Chain Selection

Each region $r \in \mathcal{R}$ is endowed with a discrete set of candidate chains $\mathcal{T}_r$, characterized by distinct flight altitudes h_t and cardinalities of coverage rings $|\mathcal{K}_t|$. Lower altitudes provide finer spatial resolution at the expense of an increased number of rings and longer flight trajectories, whereas higher altitudes reduce operational complexity but degrade resolution.

The heuristic resolves the trade-off independently for each region through a two-stage filtering and evaluation mechanism, formalized in Algorithm 2:

Algorithm 1: Multi-phase heuristic framework for the drone coverage path planning problem.

Input : Regions $\mathcal{R}$, depot v_0 , drone parameters, wind vector

Output: Set of feasible operations $\mathcal{S}$ respecting maximum endurance $W^{\max}$

// Phase 1: Pre-processing **foreach** region $r \in \mathcal{R}$ **do**

 | Select optimal chain t_r^* via proxy cost evaluation;

end

Build flat list of rings $\mathcal{K}$ from selected chains;

// Phase 2: Construction Construct region-level traveling salesperson path; expand rings in index order to form giant tour π ;

Partition π into feasible operations $\mathcal{P}$ via greedy split respecting $W^{\max}$;

// Phase 3: Improvement **for** $it \leftarrow 1$ **to** N_{coord} **do**

 | Optimize entry points across all rings via coordinate descent;

end

Execute local search: 2-opt per operation, followed by Or-opt relocation;

Build and return final solution $\mathcal{S}$;

1. **Resolution Filtering:** For a given region r , any candidate chain whose ring count $|\mathcal{K}_t|$ falls strictly below the maximum available resolution among $\mathcal{T}_r$ is pruned.
2. **Proxy Cost Minimization:** From the surviving candidates, the chain minimizing the proxy objective function $c(t)$ is selected:

$$c(t) = \sum_{\kappa \in \mathcal{K}_t} \text{perim}(\kappa) + \gamma(d_{xy}(\text{centroid}(r), v_0) + h_t),$$

where $\gamma = 0.5$ balances internal inspection effort against transit overhead and altitude energy penalties.

5.1.2. Ring Geometry and Parametrization

Once the optimal chain is fixed for every region, all selected rings are aggregated into a globally indexed sequence $\mathcal{K} = \{\kappa_1, \dots, \kappa_M\}$. For each ring κ_m , we pre-compute and store:

- The spatial centroid $\bar{p}_m = (\bar{x}_m, \bar{y}_m, h_t)$.
- The total perimeter length perim_m .
- The constituent segments $s_{m,1}, \dots, s_{m,K_m}$ alongside cumulative arc-lengths $\Lambda_m(0), \dots, \Lambda_m(K_m)$.
- An initial entry point ξ_m located at the first vertex of the ring, parameterized by an arc-length coordinate $\lambda_m \in [0, \text{perim}_m]$.

The spatial coordinates corresponding to any arc-length λ along ring κ_m are retrieved via linear interpolation across the enclosing segment k ($\Lambda_m(k-1) \leq \lambda < \Lambda_m(k)$):

$$P(\lambda) = Q_{m,k}^{\text{start}} + \frac{\lambda - \Lambda_m(k-1)}{\ell_{m,k}}(Q_{m,k}^{\text{end}} - Q_{m,k}^{\text{start}}),$$

Algorithm 2: Chain selection per region.

Input : Regions $\mathcal{R}$, depot v_0 , candidate chains $\{\mathcal{T}_r\}_{r \in \mathcal{R}}$

Output: Chain selection mapping $s : \mathcal{R} \rightarrow \mathbb{N}$

```
foreach region  $r \in \mathcal{R}$  do
     $t^* \leftarrow 0, b \leftarrow \infty$  ;
    foreach chain  $t \in \mathcal{T}_r$  do
        if  $|\mathcal{K}_t| < \max_{t' \in \mathcal{T}_r} |\mathcal{K}_{t'}|$  then
            continue
        end
         $c \leftarrow \sum_{\kappa \in \mathcal{K}_t} \text{perim}(\kappa) + \gamma(d_{xy}(\text{centroid}(r), v_0) + h_t)$  ;
        if  $c < b$  then
             $t^* \leftarrow t, b \leftarrow c$ 
        end
    end
     $s[r] \leftarrow t^*$ 
end
```

where $\ell_{m,k}$ denotes the Euclidean length of segment k . This parametric mapping underpins both the entry-point refinement (§5.3.1) and the precise energy evaluation model (§5.2.2).

5.2. Phase 2: Tour Construction

5.2.1. Region-Level Ordering and Giant Tour Generation

To determine the visitation sequence π across the M rings, we exploit the hierarchical structure of the problem. Because concentric rings belonging to the same chain share a common centroid and altitude, their internal sequencing exerts a negligible impact on transit costs. Consequently, the routing problem is decoupled into two tiers:

1. **Inter-Region Routing:** A greedy nearest-neighbor heuristic starting from the depot v_0 is executed over the 3D Euclidean centroids of the regions, yielding an ordered permutation σ of $\mathcal{R}$.
2. **Intra-Region Expansion:** For each region in sequence σ , its constituent rings are appended to the global giant tour π in increasing order of their indices (from innermost to outermost).

5.2.2. Greedy Split into Feasible Operations

Because the aggregate length of the giant tour π invariably violates the maximum drone endurance threshold $W^{\max}$, π must be partitioned into a collection of disjoint operations $\mathcal{P}$. We implement a linear-time greedy split procedure, described in Algorithm 3. Rings are sequentially accumulated into the current operation as long as the total energy—comprising departure from the depot, traversal of accumulated ring perimeters, and return to the depot—does not exceed $W^{\max} + \varepsilon$.

Algorithm 3: Greedy split procedure for endurance-feasible operations.

Input : Giant tour π , ring repository $\mathcal{K}$, maximum endurance $W^{\max}$

Output: Set of operations $\mathcal{P}$ represented as ordered lists of ring indices

$\mathcal{P} \leftarrow \emptyset, C \leftarrow \emptyset, e_{\text{cur}} \leftarrow 0$;

foreach ring index κ in π **do**

 ring $\leftarrow \mathcal{K}[\kappa]$;

$\xi_e(\kappa) \leftarrow$ energy (depot $\rightarrow$ ring) if $C = \emptyset$, else energy (prev ring $\rightarrow$ ring) ;

$r_e \leftarrow$ perim(ring) ;

$b_e \leftarrow$ energy (ring $\rightarrow$ depot) ;

if $e_{\text{cur}} + \xi_e(\kappa) + r_e + b_e \leq W^{\max} + \varepsilon$ **then**

 append κ to C ;

$e_{\text{cur}} += \xi_e(\kappa) + r_e$;

end

else

if $C \neq \emptyset$ **then**

$\mathcal{P} \leftarrow \mathcal{P} \cup \{C\}$

end

$C \leftarrow \{\kappa\}$;

$e_{\text{cur}} \leftarrow$ energy(depot $\rightarrow$ ring) + r_e ;

end

end

if $C \neq \emptyset$ **then**

$\mathcal{P} \leftarrow \mathcal{P} \cup \{C\}$

end

Physical Energy Accounting Model.. Both the split and improvement phases evaluate route feasibility via an analytical energy consumption model derived from aerodynamic drag principles ($w = \frac{1}{2}c_d A \rho v d$). Parameter configurations are detailed in Table 9.

- **Depot Transit Legs:** The energy required to travel from the depot to a target region r hosting ring ξ at altitude ξ_z incorporates wind-induced groundspeed variations:

$$E_{\text{depot} \rightarrow r}(\xi) = E_{\text{xy}}(r) \cdot d_{\text{xy}}(v_0, \xi) + E_z \cdot \xi_z, \quad E_{r \rightarrow \text{depot}}(\xi) = E_{\text{xy}}(r) \cdot d_{\text{xy}}(\xi, v_0),$$

where the effective groundspeed accounts for headwind/tailwind vectors:

$$\nu_r = \|v_{\text{cruise}} \cdot \hat{u}(v_0 \rightarrow \bar{p}_r) - w \cdot \hat{w}\|_2.$$

The respective horizontal and vertical drag coefficients are given by $E_{\text{xy}}(r) = \frac{1}{2}c_d A \rho_0 \nu_r$ and $E_z = \frac{1}{2}c_d A \rho_0 v_{\text{vert}}$.

- **Inter-Ring Transit Legs:** Energy expenditures between consecutive rings a and b are modeled via a simplified Manhattan metric consistent with the exact formulation:

$$E(a, b) = d_{\text{xy}}(a, b) + d_z(a, b).$$

5.3. Phase 3: Tour Improvement

5.3.1. Entry-Point Optimization

The precise location where the drone interfaces with a ring κ dictates the transitional overhead. Given a ring preceded by node p and succeeded by node q within the same operation, the optimal entry point minimizes the connecting distance $\|p - \xi\|_2 + \|\xi - q\|_2$.

We optimize entry points iteratively using coordinate descent. Initialized at the first vertex of the ring, each ring's entry point is updated by sampling N_{samples} candidate positions uniformly along its perimeter and selecting the coordinate that minimizes the aggregate transit distance to neighboring nodes. This procedure cycles for N_{coord} passes.

5.3.2. Local Search Metaheuristics

To escape local optima generated by the constructive phase, we implement two complementary local-search operators:

- **Intra-Operation 2-Opt:** Within each operation, every pair of non-adjacent edges (a, b) and (c, d) is evaluated for reversal. Moves yielding a strict improvement in the exact energy model are accepted. The procedure executes up to $N_{\text{passes}}^{\text{op}}$ passes.
- **Inter-Operation Or-opt:** To balance workloads across operations, individual rings are relocated between distinct operations o_s and o_d . A relocation of ring κ to position i in o_d is executed if endurance feasibility is maintained in both operations and total energy cost decreases:

$$E(o_s \setminus \{\kappa\}) \leq W^{\max}, \quad E(o_d \cup \{\kappa\} \text{ at } i) \leq W^{\max}, \quad E(o_s \setminus \{\kappa\}) + E(o_d \cup \{\kappa\} \text{ at } i) < E(o_s) + E(o_d).$$

Following any successful Or-opt move, entry points are locally re-optimized via $N_{\text{coord}}^{\text{reopt}}$ coordinate-descent passes.

5.3.3. Worked example: tracing the three phases

Table 8 follows the heuristic step by step on a small instance (3 regions, 7 rings, calm wind, $W^{\max} = 145.1$), with the exact intermediate values produced by the code. Phase 1 picks the low-altitude chain everywhere: perimeters are identical across chains, so the $0.5 h_t$ term decides (e.g. R0: $63.8 < 83.8$). Phase 2 tours $R0 \rightarrow R2 \rightarrow R1$, 2-opt reverses the tail (R1 before R2, rings outside-in), and the greedy split pays two extra depot returns ($290.2 \rightarrow 316.8$) to gain feasibility in 3 operations. Phase 3 moves all 7 entry points toward their neighbours ($316.8 \rightarrow 311.3$) and then relocates ring R1r2 from op1 to the front of op3 via or-opt ($311.3 \rightarrow 289.9$). The narrative is fully monotone because every phase optimizes the same internal proxy; the final row re-evaluates the tour leg by leg in metres, the units of the MIQP objective.

5.4. Warm-Start Integration

The final heuristic solution $\mathcal{S}$ is translated into structural variable assignments for the exact MIQP solver, initializing:

1. Chain selection variables α_t ,
2. Depot-ring allocation indicators μ_v^{ts} and γ_v ,

Table 8: Heuristic trace on a 3-region, 7-ring instance (seed 6, calm wind, $W^{\max} = 145.1$). $RXrY$ = ring Y of region X . Phase energies use the heuristic’s internal proxy (wind-weighted depot legs + ring perimeters + transit distances); the final row is the tour length in metres.

Phase	Decision	Proxy E
P1 chain sel.	$R0, R1, R2 \rightarrow$ chain 0 ($h=20$ m); proxies 63.8<83.8, 151.0<171.0, 62.5<82.5	—
P2 nearest neigh.	order $R0r0, R0r1, R2r0, R2r1, R1r0,$ $R1r1, R1r2$	294.3
P2 2-opt	reversed tail: $R0r0, R0r1, R1r2, R1r1,$ $R1r0, R2r1, R2r0$	290.2
P2 greedy split	op1 [$R0r0, R0r1, R1r2$] 131.4; op2 [$R1r1,$ $R1r0$] 129.9; op3 [$R2r1, R2r0$] 55.5	316.8
P3 coord-descent	7 entry points moved toward neighbour- ing visits	311.3
P3 or-opt	relocate $R1r2$: op1 $\rightarrow$ front of op3	289.9
Final	3 operations; tour length 573.9 m	—

Table 9: Parameter configuration of the heuristic framework (implemented in `heuristic.py`).

Parameter	Value	Algorithmic Stage	Description
N_{samples}	60	§5.1.2, §5.3.1	Perimeter sample resolution for entry search
N_{coord}	8	§5.3.1	Coordinate-descent passes for entry optimization
$N_{\text{coord}}^{\text{reopt}}$	3	§5.3.2	Re-optimization passes following Or-opt moves
$N_{\text{passes}}^{\text{op}}$	10	§5.3.2	Maximum 2-opt iterations per operation
ε	10^{-6}	§5.2.2, §5.3.2	Numerical endurance feasibility tolerance
η	10^{-9}	§5.3.2	Minimum improvement threshold for local search
γ	0.5	§5.1.1	Objective weight balancing travel cost in chain selection

3. Spatial coordinate variables P_x, P_y, P_z ,

4. Operation activation flags ζ^o and routing edge variables $x_{vv'}^o$.

This complete warm-starting strategy accelerates convergence, reducing the time required to locate the first feasible branch-and-bound incumbent by one to two orders of magnitude.

6. Computational Experiments

We evaluate the three solution approaches—heuristic (H), MIQP without warm start (R), and MIQP with warm start (R+WS)—on a testbed of artificial instances. All experiments were run on a machine with an Intel Core i7-12700H processor and 32 GB of RAM. The MIQP models were solved with Gurobi 11.0 (MIPGap=0, time limit 1800 s). The heuristic was implemented in Python 3.12 using NumPy.

6.1. Instance generation

Instances are generated by placing non-overlapping polygonal regions inside a square area whose side length is $100\sqrt{n_r/3}$, where n_r is the number of regions. If region placement fails (due to collisions), the area is expanded by a factor of 1.25 and retried up to 50 times. Each region is assigned a set of polygonal chains at the heights specified by the height level (Table 10), and each chain contains concentric rings that define the coverage pattern. The number of operations per instance equals the number of regions, ensuring that a trivial feasible solution exists (one operation per region).

Table 10: Height levels (in meters) used for each region configuration.

Height level	Heights (m)
1	30
2	20, 50
3	15, 35, 60

6.2. Instance parameters

The testbed consists of all combinations of:

- Region counts $n_r \in \{1, 2, 3, 5, 8, 10\}$;
- Height levels $\{1, 2, 3\}$ as defined in Table 10;
- Random seeds $\{0, 1, 2, 3, 4\}$ for the instance generator;
- Six endurance levels derived from a calibration phase (see below).

This yields $6 \times 3 \times 5 \times 6 = 540$ instances. Each instance is solved by all three methods, giving 1620 solver runs.

6.3. Endurance calibration

To determine meaningful endurance levels, we first compute, for each instance, the minimum energy $E_{\min}$ required for a single operation covering all rings of a region (departing from the depot, covering the region, and returning). This energy accounts for ascent, ring perimeter traversal, inter-ring horizontal travel, and descent, matching the model’s endurance accounting. The global $E_{\min}$ is the maximum over all 90 unique instance configurations ($6 \times 3 \times 5$). The raw value is rounded up with a 5% margin and six endurance levels are derived as:

$$E_\ell = \lceil E_{\min}^{\text{rounded}} \times f_\ell \rceil, \quad f_\ell \in \{1.0, 1.5, 2.0, 3.0, 5.0, 8.0\}.$$

Table 11: Calibrated endurance levels.

Factor	Raw	Rounded
$E_{\min}$ (raw)	245.93	–
$E_{\min}$ (rounded)	–	300
1.0	300	300
1.5	450	450
2.0	600	600
3.0	900	900
5.0	1500	1500
8.0	2400	2400

6.4. Solver configuration

The MIQP solver (Gurobi) is configured with the following parameters:

- MIPGap = 0 (stop only when optimality is proven);
- Time limit = 1800 s per run;
- Threads = 1;
- Heuristics = 0.1.

For the warm-start variant, the heuristic solution is provided as a start vector via Gurobi’s **Start** attribute on all variable categories (α , μ , γ , λ , x , y , ζ , k , P , u).

The heuristic solver (standalone) uses the parameters listed in Table 9.

6.5. Results

We report for each instance the objective value, solve time, and MIP gap (when available) for all three methods. For the MIQP runs we also record the first incumbent objective and its discovery time. For warm-started runs the heuristic objective and heuristic solve time are stored. Both sweeps are complete on the full testbed (540 configs $\times$ 3 methods = 1620 runs each: heuristic, exact, exact warm-started; Gurobi 11.0, TL=1800 s, MIPGap=0; duplicate re-runs in the vertex log are deduplicated keeping the last row). All figures and tables below are generated from `compare_results/results.csv` and `compare_results_edges/results.csv`.

6.6. Discussion

The aggregated results (Tables 12–13 and Figs. 6–10) confirm three patterns. First, the heuristic is consistently feasible in < 0.6 s (mean 0.03 s for $n_r = 1$ up to 0.54 s for $n_r = 10$) and stays within 1.8–7.8% of the best exact mean objective at every scale (Table 15). Second, warm-starting (R+WS, E+WS) reduces the mean time to the first feasible incumbent by one to two orders of magnitude, especially for tight endurances (Fig. 10); the effect is strongest for the Edges-Approach formulation, whose cold runs need 45 s ($n_r = 2$) to 1101 s ($n_r = 10$) for the first incumbent against 0.00 s and 7.7 s warm-started. For $n_r = 10$ the mean MIP gap (counting unsolved runs at 100%) drops from 69.5% (R) to 46.7% (R+WS) at the 1800 s limit.

Table 12: Aggregated results for the Vertex-Approach (MTZ) formulation. Mean objective and solve time averaged over height levels ($n_h = 1, 2, 3$), endurance levels ($E = 300\text{--}2400\text{ J}$) and 5 seeds. MIP gap at the 1800 s limit, counting runs with no solution as 100% gap. H = heuristic standalone.

n_r	H		R (Vertex-Approach)			R +WS (Vertex-Approach+WS)		
	Obj.	Time (s)	Obj.	Time (s)	Gap (%)	Obj.	Time (s)	Gap (%)
1	216.0	0.03	210.1	111.5	1.3	210.4	112.9	2.4
2	375.0	0.06	364.3	454.3	4.2	363.8	436.6	4.0
3	517.0	0.10	485.9	1088.8	8.0	485.8	1094.0	8.3
5	989.2	0.21	992.4	1790.3	26.8	943.4	1800.1	24.8
8	1676.2	0.41	2163.7	1800.2	55.5	1647.5	1800.2	39.2
10	2175.2	0.54	3283.6	1800.3	69.5	2017.2	1800.3	46.7

Table 13: Vertex-Approach (MTZ) vs Edges-Approach (DFJ lazy) on the full testbed: 90 configs per n_r ($n_h \times E \times \text{seeds}$), 1620 runs per formulation. Objective/time are means over runs that produced a solution; n = solved/total runs for cold Edges-Approach (it finds no solution within 1800 s on most $n_r \geq 5$ configs).

n_r	Vertex-Approach		Edges-Approach		Vertex-Approach+WS		Edges-Approach+WS		n (Edges-Approach)
	Obj.	Time	Obj.	Time	Obj.	Time	Obj.	Time	solved
1	210.1	111.5	210.1	172.6	210.4	112.9	210.1	165.7	89/90
2	364.3	454.3	364.1	872.5	363.8	436.6	363.8	833.4	88/90
3	485.9	1088.8	490.7	1380.8	485.8	1094.0	487.3	1406.4	81/90
5	992.4	1790.3	832.7	1803.4	943.4	1800.1	952.9	1801.7	40/90
8	2163.7	1800.2	1094.0	1806.5	1647.5	1800.2	1649.5	1801.9	11/90
10	3283.6	1800.3	1170.9	1805.1	2017.2	1800.3	2032.6	1802.5	2/90

Third, the head-to-head comparison must account for censoring: plain means overstate cold Edges-Approach, which leaves most large instances unsolved (only 40/90 solved at $n_r = 5$, 11/90 at $n_r = 8$ and 2/90 at $n_r = 10$; see the solved counts in Table 13 and the plateaus of Fig. 9). On the configs where both formulations produced a solution, they are equivalent in objective (parity scatter of Fig. 8, mean ratio 1.023 cold on 311 configs, 1.003 warm on 528); with warm-start the two formulations agree within 1% even on the largest instances ($n_r = 8$: 1649.5 vs. 1647.5; $n_r = 10$: 2032.6 vs. 2017.2). Counting unsolved runs as failures, the Vertex-Approach formulation solves 509/540 configs (94%) cold against 311/540 (58%) for the Edges-Approach one, and 528/540 vs. 530/540 warm-started. The Edges-Approach model uses $\approx 2\text{--}5\%$ fewer variables but incurs DFJ lazy-cut overhead: it is markedly slower without warm-start on small instances (172.6 s vs. 85.1 s at $n_r = 1$; 872.5 s vs. 454.3 s at $n_r = 2$), while on large instances both hit the time limit. Warm-starting restores feasibility almost everywhere and moves the gap mass left for both formulations (Fig. 9: fraction under 50% gap from 57% to 90% for edges).

Table 14 isolates the endurance effect for $n_r = 3$: with $E = 300\text{ J}$ the mean total distance is 525 m and strictly decreases to 474 m at $E \geq 600\text{ J}$, reflecting fewer depot returns; beyond 600 J the curves flatten, indicating that the instance becomes endurance-unconstrained for that size. The gap pattern is non-monotonic in E because tight E yields many small operations that are easier to sequence, while intermediate E creates harder packing decisions.

Table 14: Effect of endurance E for $n_r = 3$ (averaged over $n_h = 1, 2, 3$ and 5 seeds). Tight E implies more operations and higher total distance.

E (J)	H		R			$R+WS$		
	Obj.	Time	Obj.	Gap	Time	Obj.	Gap	Time
300	570.8	0.10	524.7	9.5	1122.7	525.8	9.5	1066.8
450	528.3	0.10	492.2	9.7	1121.8	492.2	10.8	1120.2
600	500.7	0.09	474.3	6.5	1062.7	474.3	7.6	1143.7
900	500.7	0.09	474.5	7.5	1070.7	474.3	7.4	1110.3
1500	500.7	0.10	475.2	7.0	1037.1	474.3	7.4	1035.5
2400	500.7	0.10	474.5	8.1	1117.7	474.3	7.1	1087.4

Table 15: Heuristic solution quality: mean heuristic objective vs. best exact mean objective per region count (averaged over n_h , E and seeds; exact = min over the four MIQP runs per instance).

n_r	Heur. obj.	Best exact obj.	Ratio
1	216.0	210.1	1.028
2	375.0	363.8	1.031
3	517.0	485.7	1.065
5	989.2	941.9	1.050
8	1676.2	1646.6	1.018
10	2175.2	2017.2	1.078

6.7. Limitations and reproducibility

Three limitations should be kept in mind. First, all experiments use artificial instances (regular polygons on a grid, Gurobi with an 1800s limit); no real mission data is tested. Second, the wind-aware extension assumes a single constant wind direction with a power-law speed profile, and fixes inter-region leg directions offline via region centroids—a good approximation for compact regions but not for elongated ones. Third, the K -split and wind-aware models are validated on selected showcase instances, not on the full testbed, and the optimal tours reported beyond $K = 3$ are solver-limited rather than proven optimal.

All experiments are reproducible by seed: instance generation, the vertex/edge MIQPs, the heuristic, the K -sweep, and the wind-aware demos are implemented in the accompanying Python/Gurobi codebase, and every table and figure in this paper is generated from the raw solver logs by versioned scripts.

7. Conclusions and future work

We have presented a mixed-integer quadratic programming formulation for the drone coverage path planning problem with altitude selection, variable entry and exit points, and operation splitting under endurance constraints. Unlike previous two-stage approaches that

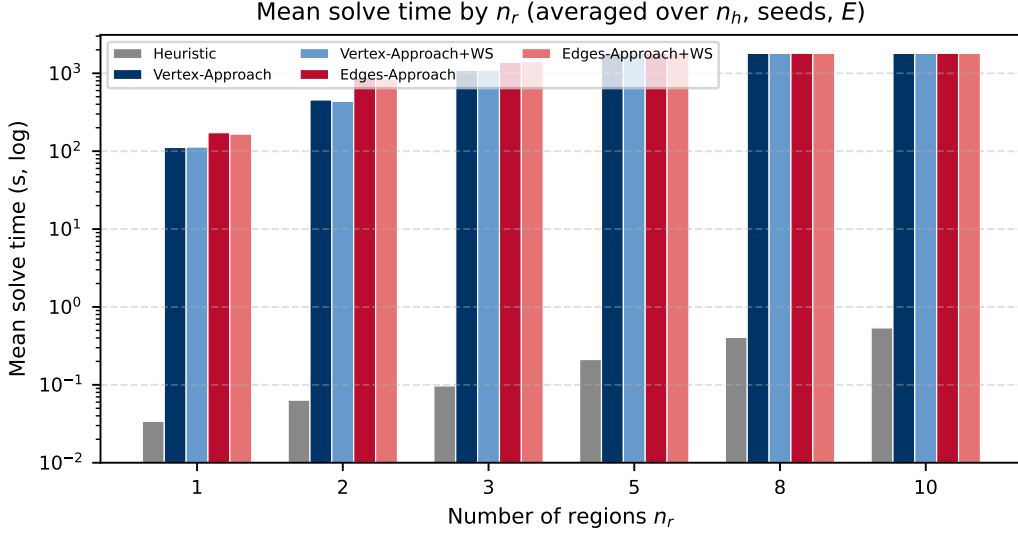


Figure 6: Mean solve time (log scale) vs. number of regions n_r , averaged over n_h , E and seeds. Vertex-Approach (MTZ) vs. Edges-Approach (DFJ lazy) with and without warm-start, plus heuristic, on the full testbed.

decouple intra-region and inter-region planning, our formulation allows the drone to interrupt coverage of a region and move to another, potentially reducing total energy consumption.

To address the computational complexity of the MIQP, we developed a multi-phase heuristic that constructs feasible solutions through chain selection, region-level tour construction, greedy endurance-aware splitting, coordinate-descent entry-point optimization, and intra/inter-operation local search. The heuristic runs in well under a second on all tested instances and provides an effective warm start for the exact solver.

On the completed testbed (540 configs per formulation) the heuristic is always feasible and warm-starting consistently improves both time-to-first-incumbent and final gap; the Edges-Approach formulation is equivalent in solution quality once warm-started (mean warm ratio 1.003; agreement within 1% on the largest instances) but strictly dominated without warm-start, where it solves only 311/540 configs (58%) against 509/540 (94%) for the Vertex-Approach model. The heuristic stays within 1.8–7.8% of the best exact mean objective at every scale (Table 15).

Two extensions broaden the model. Splitting each coverage ring into K arcs removes the coincident entry/exit restriction: one split point per ring ($K = 2$) already improves the single-loop optimum by 1.5–25% across the tested instances, a second cut helps on larger cases, and nothing is gained beyond $K = 3$ within solver reach—while the variable count grows from $\approx 3k$ ($K = 1$) to $\approx 66k$ ($K = 5$) on the demo instance (Table 5, Fig. 11b). The wind-aware extension re-expresses energy in real joules with exact per-heading costs on rings and centroid-direction costs on transits; at tight and loose budgets the optimal tours are identical under calm air and 8 m/s wind (Table 7), and the $K = 2$ wind-aware optimum (652.1m, Fig. 5) confirms that splits remain beneficial under real energy accounting.

Several directions for future work can be identified:

- Extending the model to multiple heterogeneous drones, introducing coordination and

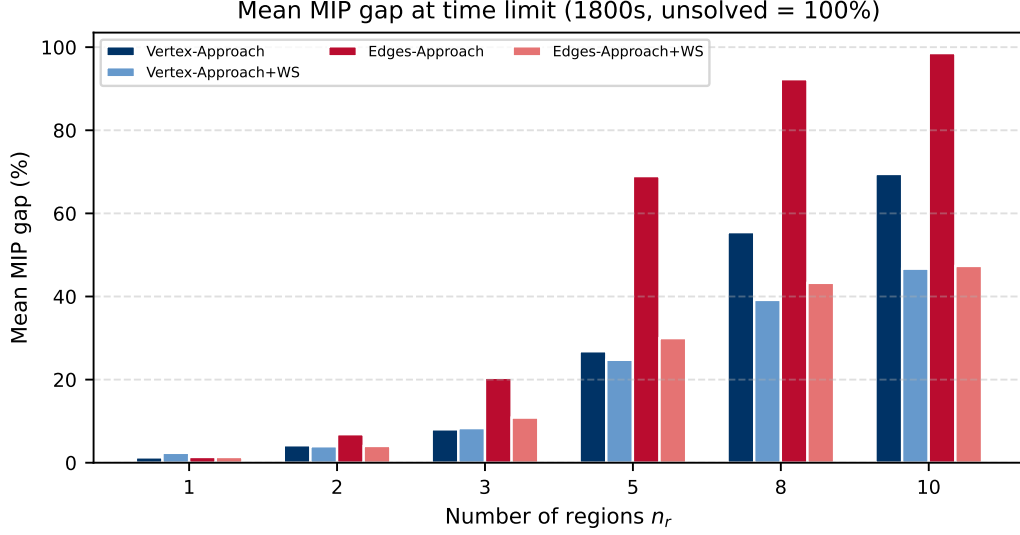


Figure 7: Mean MIP gap at the 1800s limit (runs with no solution counted at 100% gap). Both formulations prove optimality for $n_r = 1$; the gap grows with n_r . Warm-start reduces the gap for large instances (e.g. $n_r = 10$: Vertex-Approach 69.5% $\rightarrow$ Vertex-Approach+WS 46.7%). Cold Edges-Approach finds almost nothing on large instances, hence its near-100% bars at $n_r = 8, 10$.

scheduling constraints;

- Benchmarking the K -split and wind-aware models on the full testbed and on realistic instances derived from actual aerial survey or precision agriculture missions;
- Developing more sophisticated metaheuristics (e.g. variable neighbourhood search, large neighbourhood search) to improve solution quality further;
- Performing a polyhedral comparison of the MTZ and DFJ lazy relaxations to explain the primal strength of the Edges-Approach formulation on large instances.

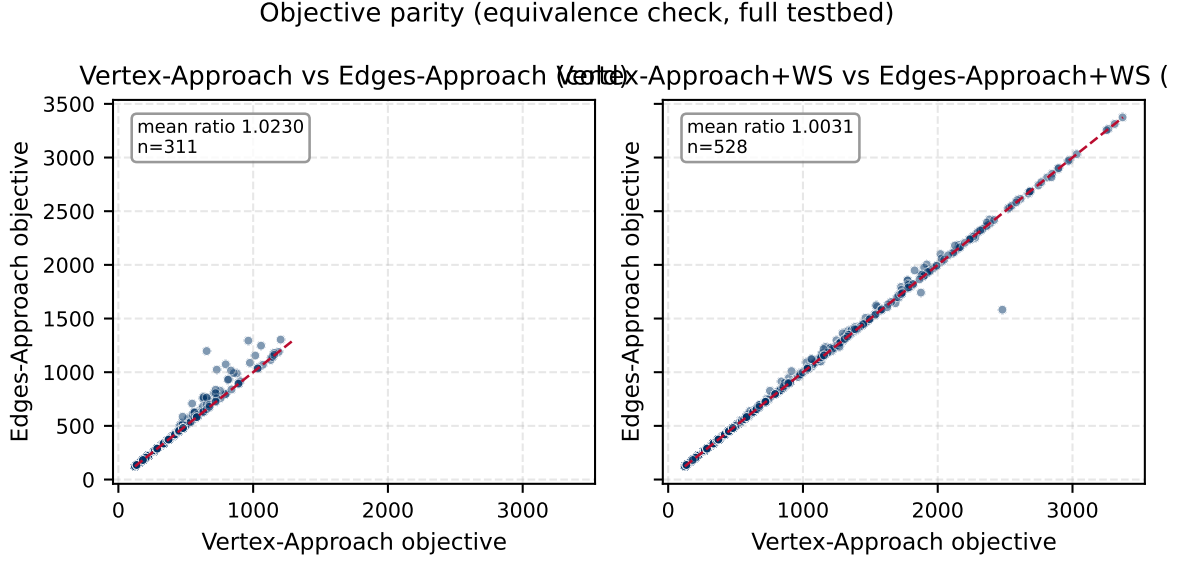


Figure 8: Objective parity check: Vertex-Approach (MTZ) vs. Edges-Approach (DFJ) on the 311 (cold) / 528 (warm) configs where both formulations produced a solution. Diagonal = equivalence. Mean ratio Edges-Approach/Vertex-Approach = 1.023 (cold) and 1.003 (warm), confirming the two MIQPs are equivalent up to solver variability and time-limit effects; warm-start closes the residual gap.

References

- [1] L. Chiaraviglio, L. Amorosi, F. Malandrino, C. F. Chiasserini, C. Casetti, P. Dell’Olmo, Optimal throughput management in uav-based networks during disasters., in: IEEE Conference on Computer Communications, 2019, pp. 307–312.
- [2] Y. Shi, J. Yang, Q. Han, H. Song, H. Guo, Optimal decision-making of post-disaster emergency material scheduling based on helicopter–truck–drone collaboration, *Omega* 127 (2024) 103104.
- [3] L. Amorosi, L. Chiaraviglio, J. Galan-Jimenez, Optimal energy management of uav-based cellular networks powered by solar panels and batteries: Formulation and solutions., *IEEE Access* 7 (2019) 53698–53717.
- [4] L. Amorosi, J. Puerto, C. Valverde, An extended model of coordination of an all-terrain vehicle and a multivisit drone., *International Transactions in Operational Research* 31 (2) (2024) 780–806.
- [5] L. Amorosi, J. Puerto, C. Valverde, Coordinating drones with mothership vehicles: The mothership and drone routing problem with graphs., *Computers and Operations Research* 136 (2021) 105445.
- [6] L. Amorosi, J. Puerto, C. Valverde, A multiple-drone arc routing and mothership coordination problem., *Computers and Operations Research* 159 (2023) 106322.
- [7] F. Nexand, F. Remondino, Uav for 3d mapping applications: A review., *Appl.Geoemat.* 6 (2014) 1–15.

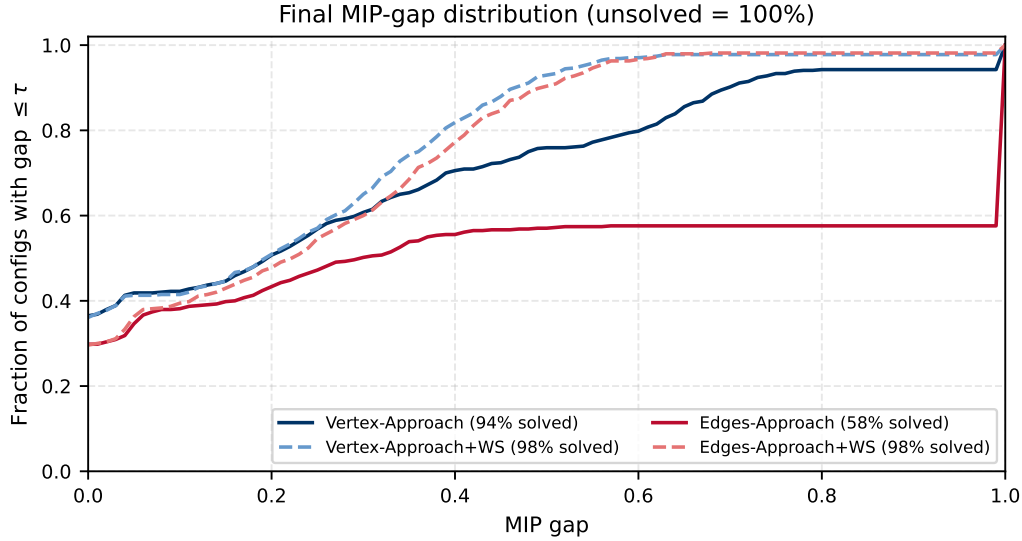


Figure 9: Distribution of final MIP gaps over the 540 configs (runs with no solution counted at 100%). Warm-starting shifts both formulations markedly left: the fraction under 50% gap rises from 76% to 93% (vertex) and from 57% to 90% (edges). Legend shows the fraction of configs solved by each method.

- [8] P. Tokekaretal, Sensor planning for a symbiotic uav and ugv system for precision agriculture., IEEE Trans.Robot. 32 (6) (2016) 1498–1511.
- [9] A. Otto, N. Agatz, J. Campbell, B. Golden, E. Pesch, Optimization approaches for civil applications of unmanned aerial vehicles (uavs) or aerial drones: A survey., Networks 72 (2018) 1–48.
- [10] S. Sonawane, K. Pal, Area-coverage path planning for unmanned aerial vehicles: A review *, in: 2023 Third International Conference on Secure Cyber Computing and Communication (ICSCCC), 2023, pp. 744–749. doi:10.1109/ICSCCC58608.2023.10176527.
- [11] X. Mu, W. Gao, X. Li, G. Li, Coverage path planning for uav based on improved back-and-forth mode, IEEE Access 11 (2023) 114840–114854. doi:10.1109/ACCESS.2023.3325483.
- [12] G. Ahmed, T. Sheltami, A. Mahmoud, Energy-efficient multi-uav multi-region coverage path planning approach, Arabian Journal for Science and Engineering 49 (2024). URL <https://doi.org/10.1007/s13369-024-09295-w>
- [13] P. Tripicchio, M. Unetti, S. D’Avella, C. A. Avizzano, Smooth coverage path planning for uavs with model predictive control trajectory tracking, Electronics 12 (10) (2023). doi:10.3390/electronics12102310. URL <https://www.mdpi.com/2079-9292/12/10/2310>
- [14] V.-C. E. V., V.-G. J. I., J. C. H.-L. J. C., A.-C. M., Coverage path planning for spraying drones, Computers & Industrial Engineering 168 (2022). doi:<https://doi.org/10.1016/j.cie.2022.108125>.

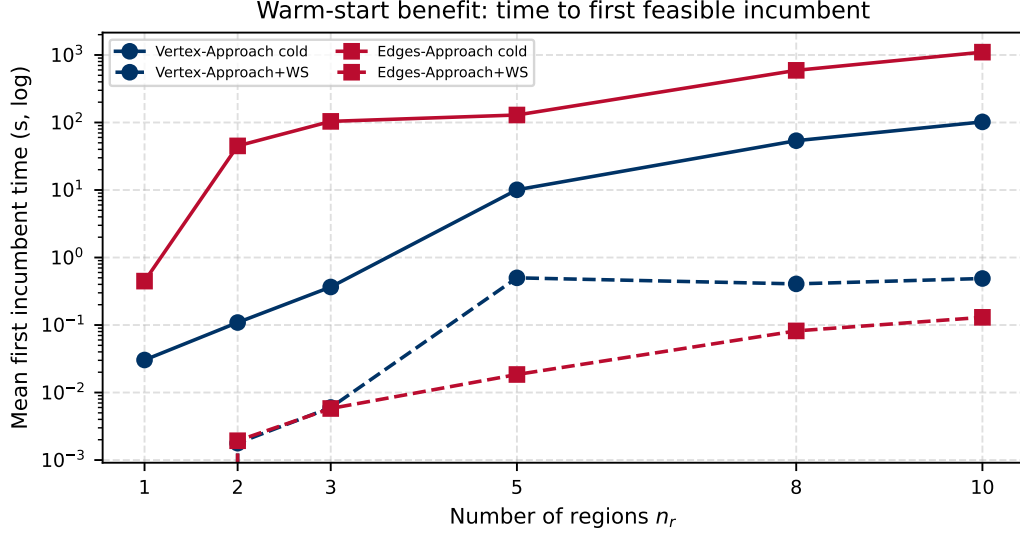


Figure 10: Time to first feasible incumbent (mean over seeds/ n_h/E). Solid = cold start, dashed = warm-start. Warm-start reduces the time by one to two orders of magnitude, especially for tight endurances; the effect is visible for both Vertex-Approach and Edges-Approach.

- [15] L. Wang, X. Zhuang, W. Zhang, J. Cheng, T. Zhang, Coverage path planning for uavs: An energy-efficient method in convex and non-convex mixed regions, *Drones* 8 (12) (2024).
URL <https://www.mdpi.com/2504-446X/8/12/776>
- [16] H. Li, R. Jia, Z. Zheng, M. Li, Energy-efficient uav trajectory design and velocity control for visual coverage of terrestrial regions, *Drones* 9 (5) (2025).
doi:10.3390/drones9050339.
URL <https://www.mdpi.com/2504-446X/9/5/339>
- [17] S. Heier, Grid integration of wind energy: onshore and offshore conversion systems, John Wiley & Sons, 2014.
- [18] C. E. Miller, A. W. Tucker, R. A. Zemlin, Integer programming formulation of traveling salesman problems, *Journal of the ACM* 7 (4) (1960) 326–329.

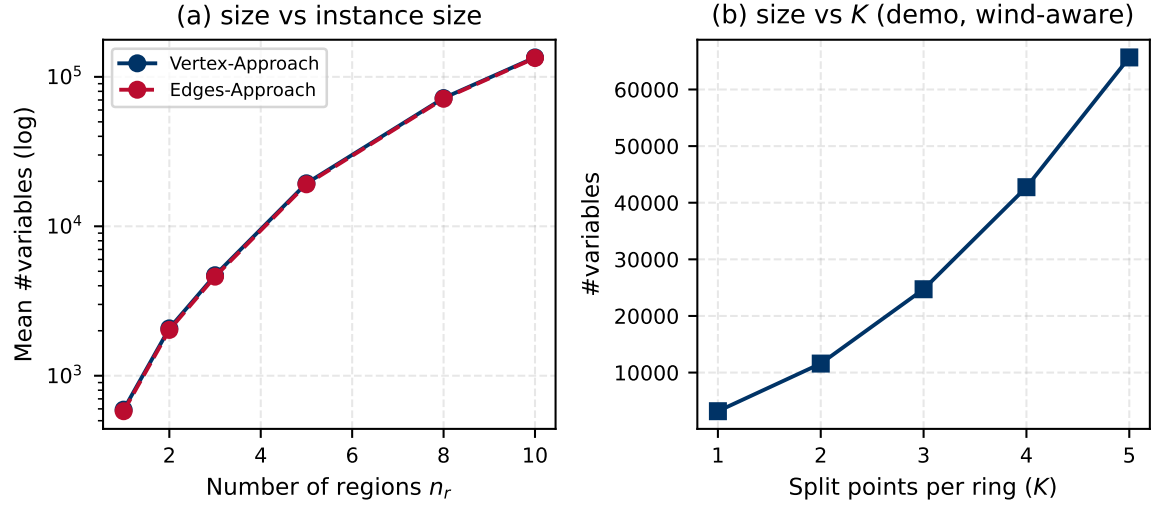


Figure 11: Model size scaling: (a) mean variable count vs. number of regions for both formulations (log scale); (b) variable count vs. split points K on the wind-aware demo. The quadratic growth in $|\mathcal{V}|$ (routing variables over vertex pairs) and the steep cost of extra split points explain the solver behaviour observed in Tables 5–12.